

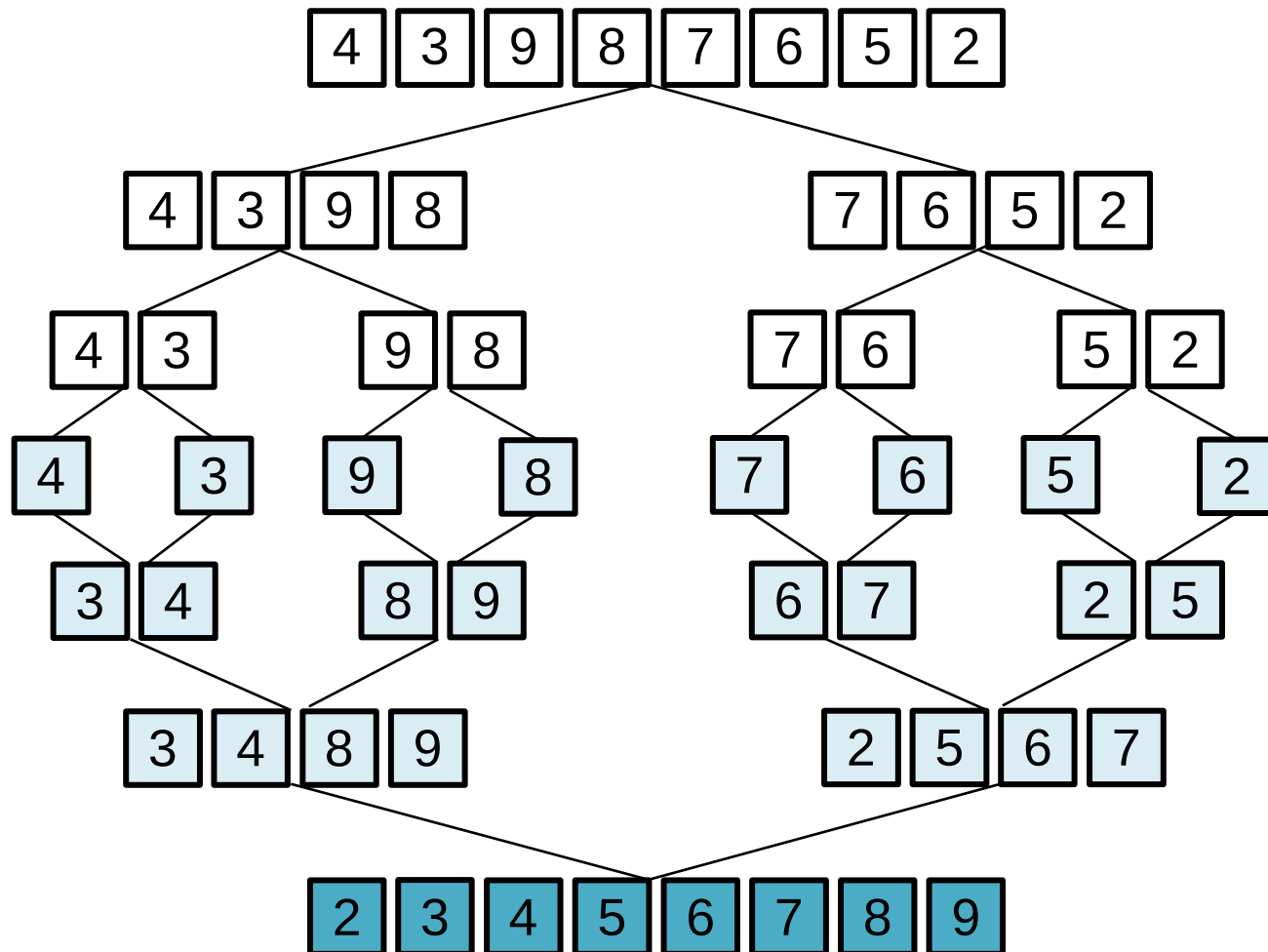
Data Structures

Lecture 16: Priority Queue & Heap

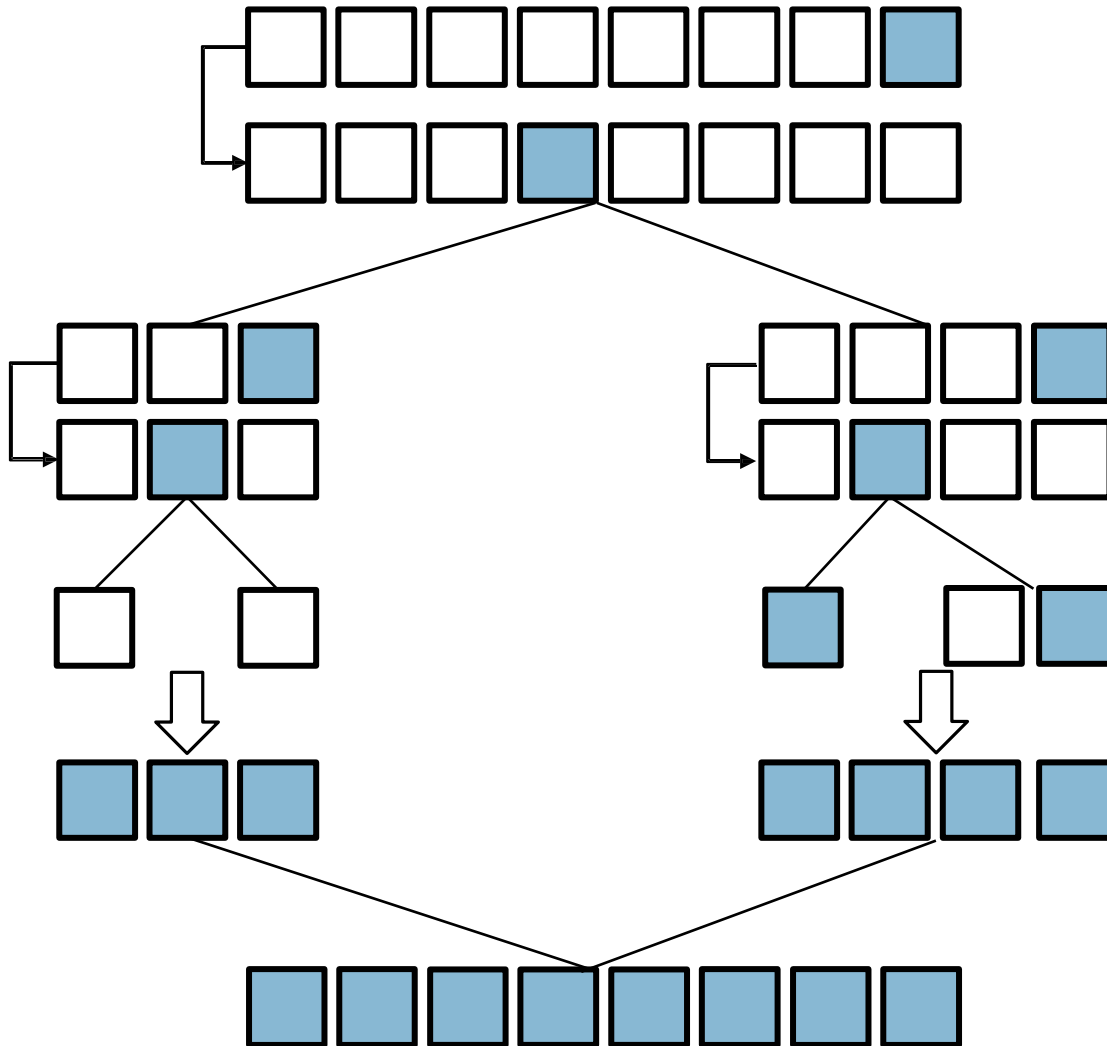
Soobin Um

May 11, 2026

Recap: Merge Sort



Recap: Quick Sort



Recap: Quick Sort vs. Merge Sort

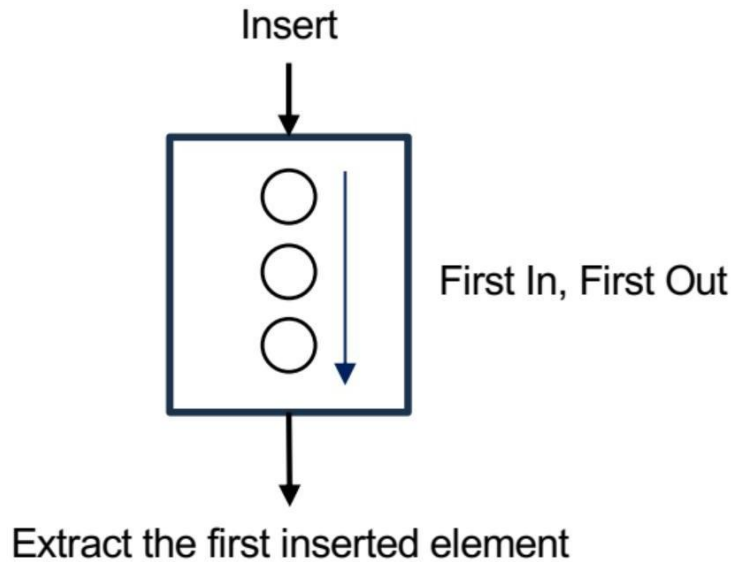
- 평균 시간 복잡도는 서로 동일
- 메모리 공간 사용 측면에선 퀵 정렬이 더 유리

		Quick Sort	Merge Sort
시간	시간 복잡도 (평균)	$O(n \log n)$	$O(n \log n)$
	시간 복잡도 (최악)	$O(n^2)$ - 피벗이 매번 최소 또는 최대값으로 선택되는 경우 - 입력 배열이 이미 거의 정렬된 상태일 경우	$O(n \log n)$
공간	메모리 공간 복잡도	$O(\log n)$ - In-place 방식으로 수행 (기존 저장 공간을 재활용) - 재귀 호출로 인한 스택 공간 (트리의 깊이) 만을 필요	$O(n)$ - 병합 과정에서 크기 n 의 보조 배열 필요 (e.g., merge의 result 배열)
	캐시 효율	높음 (연속 접근 가능)	낮음 (두 배열 왕복)

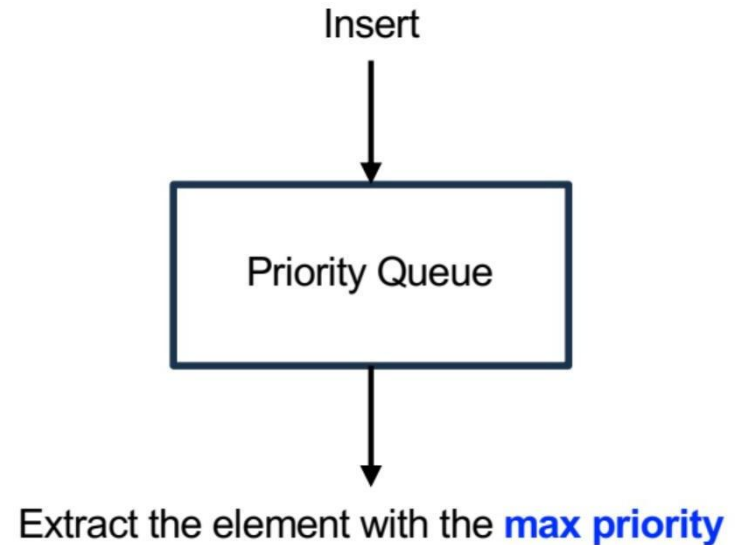
Priority Queue

Priority Queue (우선순위 큐)

- 각 원소가 우선순위 값을 가지며, 이 우선순위에 따라 원소들이 저장/처리되는 큐의 한 종류
- FIFO 구조인 일반 큐와 달리, 우선순위가 높은 원소가 낮은 원소보다 먼저 처리(dequeue)됨



일반적인 큐



우선순위 큐

사용 예시

- 멀티태스킹을 지원하는 운영체제에서의 프로세스 스케줄링
- 네트워크 패킷 처리
 - 긴급도가 높은 데이터를 우선적으로 전송
- 디ijkstra (Dijkstra) 최단 경로 알고리즘
 - 현재까지의 최단 거리 값이 가장 작은 노드를 우선적으로 꺼내야 함
- 허프만 코딩 (Huffman Coding) 데이터 압축 알고리즘
 - 문자 등장 빈도가 낮은 노드부터 우선적으로 꺼내야 함

구현 방법

- 정렬되지 않은 배열/리스트
 - 삽입: 그냥 맨 뒤에 추가 ($O(1)$)
 - 삭제: 전체를 탐색하여 가장 높은 우선순위의 원소(ex: 최대 값)을 찾아야 함 ($O(n)$) → 비효율적

구현 방법

- 정렬되지 않은 배열/리스트
 - 삽입: 그냥 맨 뒤에 추가 ($O(1)$)
 - 삭제: 전체를 탐색하여 가장 높은 우선순위의 원소(ex: 최대 값)을 찾아야 함 ($O(n)$) → 비효율적
- 정렬된 배열/리스트
 - 삽입: 정렬 상태를 유지해야 하므로, 적절한 위치를 찾아 삽입해야 함 ($O(n)$) → 비효율적
 - 삭제: 바로 삭제 가능 ($O(1)$)

구현 방법

- 정렬되지 않은 배열/리스트
 - 삽입: 그냥 맨 뒤에 추가 ($O(1)$)
 - 삭제: 전체를 탐색하여 가장 높은 우선순위의 원소(ex: 최대 값)을 찾아야 함 ($O(n)$) → 비효율적
- 정렬된 배열/리스트
 - 삽입: 정렬 상태를 유지해야 하므로, 적절한 위치를 찾아 삽입해야 함 ($O(n)$) → 비효율적
 - 삭제: 바로 삭제 가능 ($O(1)$)

우선순위 큐는 삽입과 삭제 모두가 자주 일어나는 자료 구조

→ 정렬된 또는 정렬되지 않은 배열 및 리스트는 우선순위 큐 구현에 부적합

구현 방법

- 정렬되지 않은 배열/리스트
 - 삽입: 그냥 맨 뒤에 추가 ($O(1)$)
 - 삭제: 전체를 탐색하여 가장 높은 우선순위의 원소(ex: 최대 값)을 찾아야 함 ($O(n)$) → 비효율적
- 정렬된 배열/리스트
 - 삽입: 정렬 상태를 유지해야 하므로, 적절한 위치를 찾아 삽입해야 함 ($O(n)$) → 비효율적
 - 삭제: 바로 삭제 가능 ($O(1)$)

우선순위 큐는 삽입과 삭제 모두가 자주 일어나는 자료 구조

→ 정렬된 또는 정렬되지 않은 배열 및 리스트는 우선순위 큐 구현에 부적합



트리 기반의 "힙 (Heap) 구조"를 이용
(트리 구조이므로, 삽입과 삭제 모두 $O(\log n)$ 에 가능)

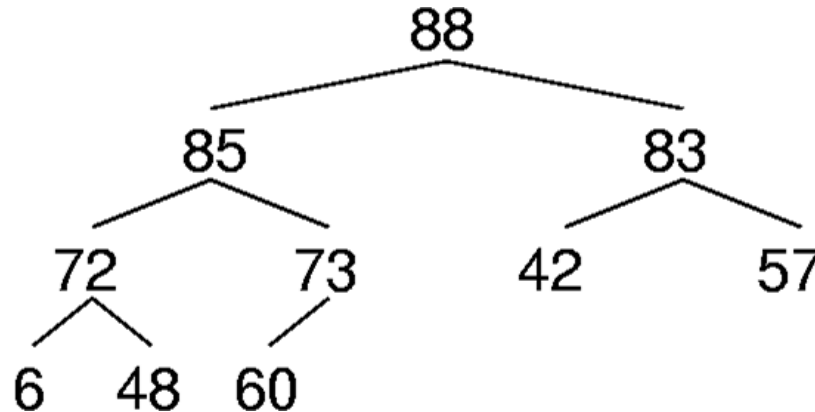
Heap

Heap (힙: 더미, 무더기)

- 힙 속성(Heap Property)을 만족하는 완전이진트리(Complete BT)

Heap (힙: 더미, 무더기)

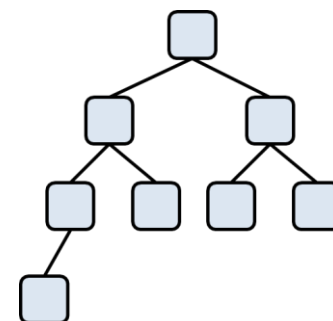
- 힙 속성(Heap Property)을 만족하는 완전이진트리(Complete BT)
 - 힙 속성
 - Max-heap: 모든 노드는 자손 노드보다 큰 값을 가짐 (→ 본 강의의 초점)
 - Min-heap: 모든 노드는 자손 노드보다 작은 값을 가짐



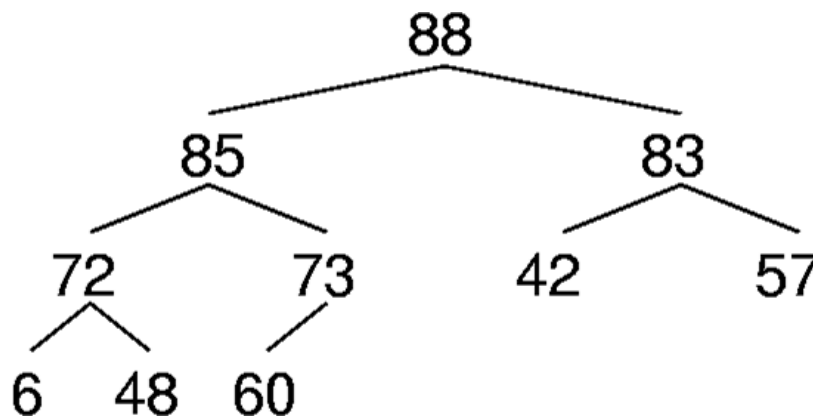
Max heap의 예

Heap (힙: 더미, 무더기)

- 힙 속성(Heap Property)을 만족하는 완전이진트리(Complete BT)
 - 힙 속성
 - Max-heap: 모든 노드는 자손 노드보다 큰 값을 가짐 (→ 본 강의의 초점)
 - Min-heap: 모든 노드는 자손 노드보다 작은 값을 가짐
 - 완전이진트리(Complete Binary Tree)
 - 마지막 레벨을 제외한 모든 레벨이 가득 차 있음
 - 마지막 레벨의 경우, 왼쪽에서부터 채워져 나감



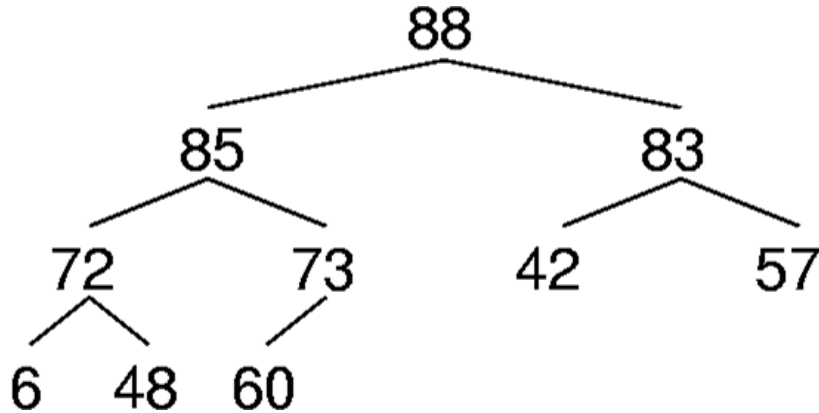
Complete Binary Tree



Max heap의 예

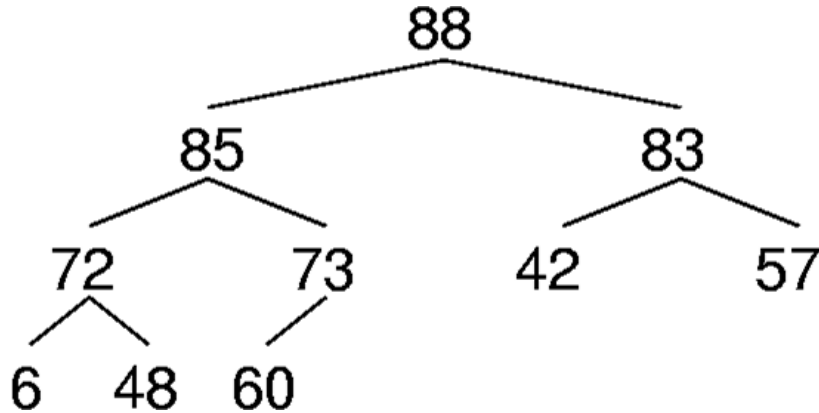
Heap as Array

- 힙은 완전이진트리이므로, 배열로 구현(또는 저장)될 수 있음



Heap as Array

- 힙은 완전이진트리이므로, 배열로 구현(또는 저장)될 수 있음



$$\text{parent}(i) = \lfloor (i - 1) / 2 \rfloor$$

$$\text{leftChild}(i) = 2i + 1 \text{ (if } 2i + 1 < n \text{)}$$

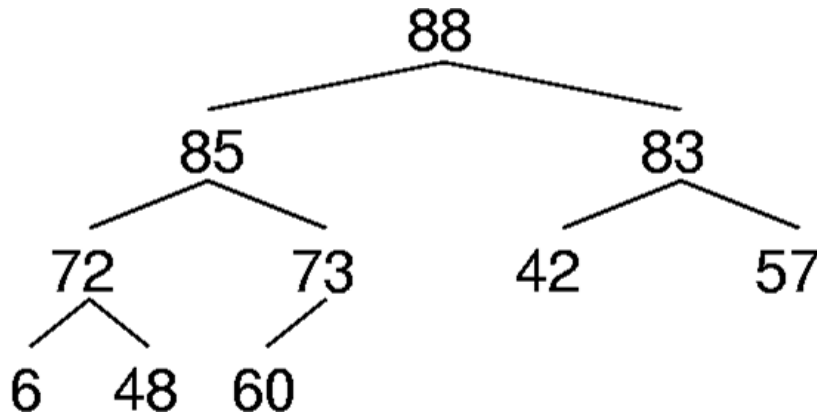
$$\text{rightChild}(i) = 2i + 2 \text{ (if } 2i + 2 < n \text{)}$$

$$\text{leftSibling}(i) = i - 1$$

$$\text{rightSibling}(i) = i + 1$$

Heap as Array

- 힙은 완전이진트리이므로, 배열로 구현(또는 저장)될 수 있음



88	85	83	72	73	42	57	6	48	60			
0	1	2	3	4	5	6	7	8	9	10	11	12

$$\begin{aligned} \text{parent}(i) &= \lfloor (i - 1) / 2 \rfloor \\ \text{leftChild}(i) &= 2i + 1 \text{ (if } 2i + 1 < n \text{)} \\ \text{rightChild}(i) &= 2i + 2 \text{ (if } 2i + 2 < n \text{)} \\ \text{leftSibling}(i) &= i - 1 \\ \text{rightSibling}(i) &= i + 1 \end{aligned}$$

Search in Heap

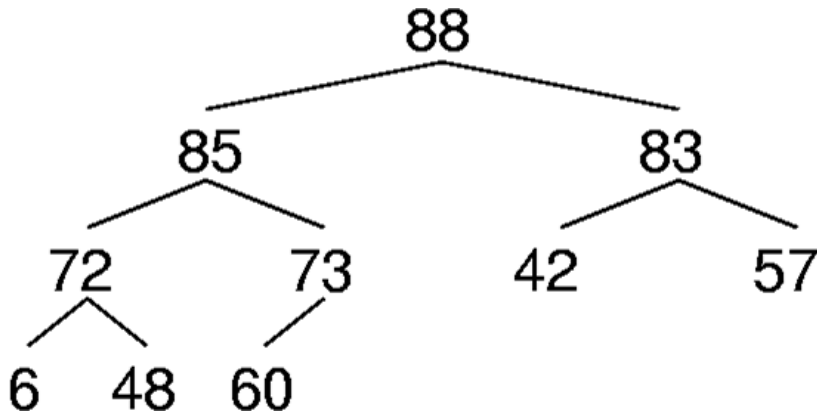
- 힙은 완전이진트리 구조 $\rightarrow$ 탐색은 $O(n)$ 의 시간 복잡도를 지님

Search in Heap

- 힙은 완전이진트리 구조 $\rightarrow$ 탐색은 $O(n)$ 의 시간 복잡도를 지님
- 힙은 루트만 정렬되어 있는, **부분 정렬 상태**의 트리 (루트의 최소 또는 최대값만 보장)
 - (참고) 이진탐색트리(BST)는 완전 정렬 상태의 트리, 탐색 $O(\log n)$

Search in Heap

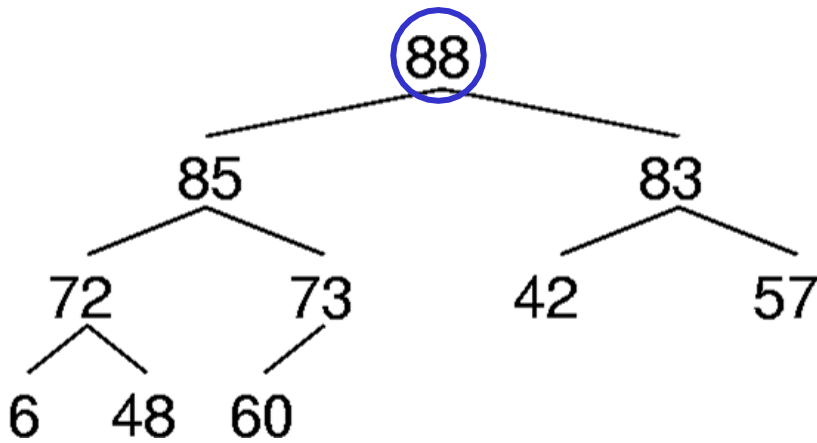
- 힙은 완전이진트리 구조 $\rightarrow$ 탐색은 $O(n)$ 의 시간 복잡도를 지님
- 힙은 루트만 정렬되어 있는, **부분 정렬 상태**의 트리 (루트의 최소 또는 최대값만 보장)
 - (참고) 이진탐색트리(BST)는 완전 정렬 상태의 트리, 탐색 $O(\log n)$



88	85	83	72	73	42	57	6	48	60			
0	1	2	3	4	5	6	7	8	9	10	11	12

Search in Heap

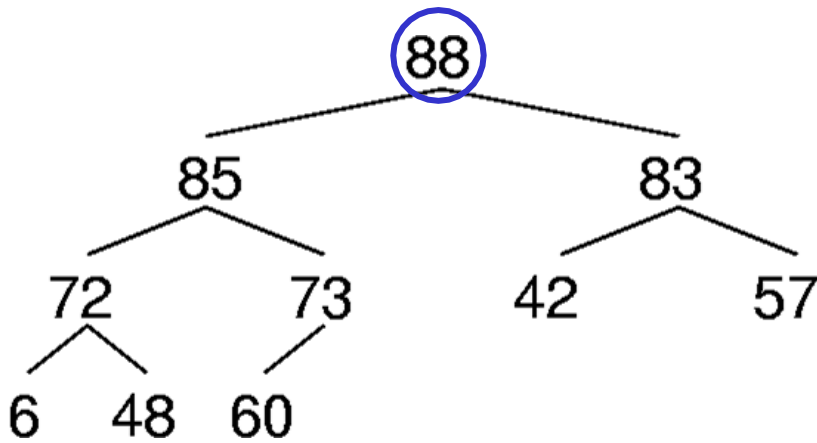
- 힙은 완전이진트리 구조 $\rightarrow$ 탐색은 $O(n)$ 의 시간 복잡도를 지님
- 힙은 루트만 정렬되어 있는, **부분 정렬 상태**의 트리 (루트의 최소 또는 최대값만 보장)
 - (참고) 이진탐색트리(BST)는 완전 정렬 상태의 트리, 탐색 $O(\log n)$



88	85	83	72	73	42	57	6	48	60			
0	1	2	3	4	5	6	7	8	9	10	11	12

Search in Heap

- 힙은 완전이진트리 구조 → 탐색은 $O(n)$ 의 시간 복잡도를 지님
- 힙은 루트만 정렬되어 있는, **부분 정렬 상태**의 트리 (루트의 최소 또는 최대값만 보장)
 - (참고) 이진탐색트리(BST)는 완전 정렬 상태의 트리, 탐색 $O(\log n)$



88	85	83	72	73	42	57	6	48	60			
0	1	2	3	4	5	6	7	8	9	10	11	12

힙은 “부분 정렬” 되어있어 BST보다 임의 원소 탐색 연산에 훨씬 더 불리
하지만 이 점 때문에, 우선순위 큐 구현에는 BST보다 더 효과적

구현: Max Heap

```
public class MaxHeap {
    private int[] Heap;    // Pointer to heap array
    private int size;      // Maximum size of heap
    private int n;         // # of things in heap

    public MaxHeap(int[] arr, int num, int max) {
        Heap = arr; n = num; size = max;
        buildHeap();
    }

    public int heapSize() { return n; }

    // Is pos a leaf position?
    public boolean isLeaf(int pos) {
        return (pos >= n / 2) && (pos < n);
    }

    // Leftchild position
    public int leftChild(int pos) {
        if (pos >= n / 2)
            throw new IllegalArgumentException("Position has no left child");
        return 2 * pos + 1;
    }

    // Rightchild position
    public int rightChild(int pos) {
        if (pos >= (n - 1) / 2)
            throw new IllegalArgumentException("Position has no right child");
        return 2 * pos + 2;
    }

    public int parent(int pos) {
        if (pos <= 0)
            throw new IllegalArgumentException("Position has no parent");
        return (pos - 1) / 2;
    }

    // (나머지 함수 - siftDown, buildHeap, removeMax, insert 등)
}
```

```
parent(i) =  $\lfloor (i - 1) / 2 \rfloor$ 
leftChild(i) =  $2i + 1$  (if  $2i + 1 < n$ )
rightChild(i) =  $2i + 2$  (if  $2i + 2 < n$ )
leftSibling(i) =  $i - 1$ 
rightSibling(i) =  $i + 1$ 
```


구현: Max Heap

```
public class MaxHeap {
    private int[] Heap;    // Pointer to heap array
    private int size;      // Maximum size of heap
    private int n;         // # of things in heap

    public MaxHeap(int[] arr, int num, int max) {
        Heap = arr; n = num; size = max;
        buildHeap();
    }

    public int heapSize() { return n; }

    // Is pos a leaf position?
    public boolean isLeaf(int pos) {
        return (pos >= n / 2) && (pos < n);
    }

    // Leftchild position
    public int leftChild(int pos) {
        if (pos >= n / 2)
            throw new IllegalArgumentException("Position has no left child");
        return 2 * pos + 1;
    }

    // Rightchild position
    public int rightChild(int pos) {
        if (pos >= (n - 1) / 2)
            throw new IllegalArgumentException("Position has no right child");
        return 2 * pos + 2;
    }

    public int parent(int pos) {
        if (pos <= 0)
            throw new IllegalArgumentException("Position has no parent");
        return (pos - 1) / 2;
    }

    // (나머지 함수 - siftDown, buildHeap, removeMax, insert 등)
}
```

```
parent(i) =  $\lfloor (i - 1) / 2 \rfloor$ 
leftChild(i) =  $2i + 1$  (if  $2i + 1 < n$ )
rightChild(i) =  $2i + 2$  (if  $2i + 2 < n$ )
leftSibling(i) =  $i - 1$ 
rightSibling(i) =  $i + 1$ 
```

Build Heap: sift down

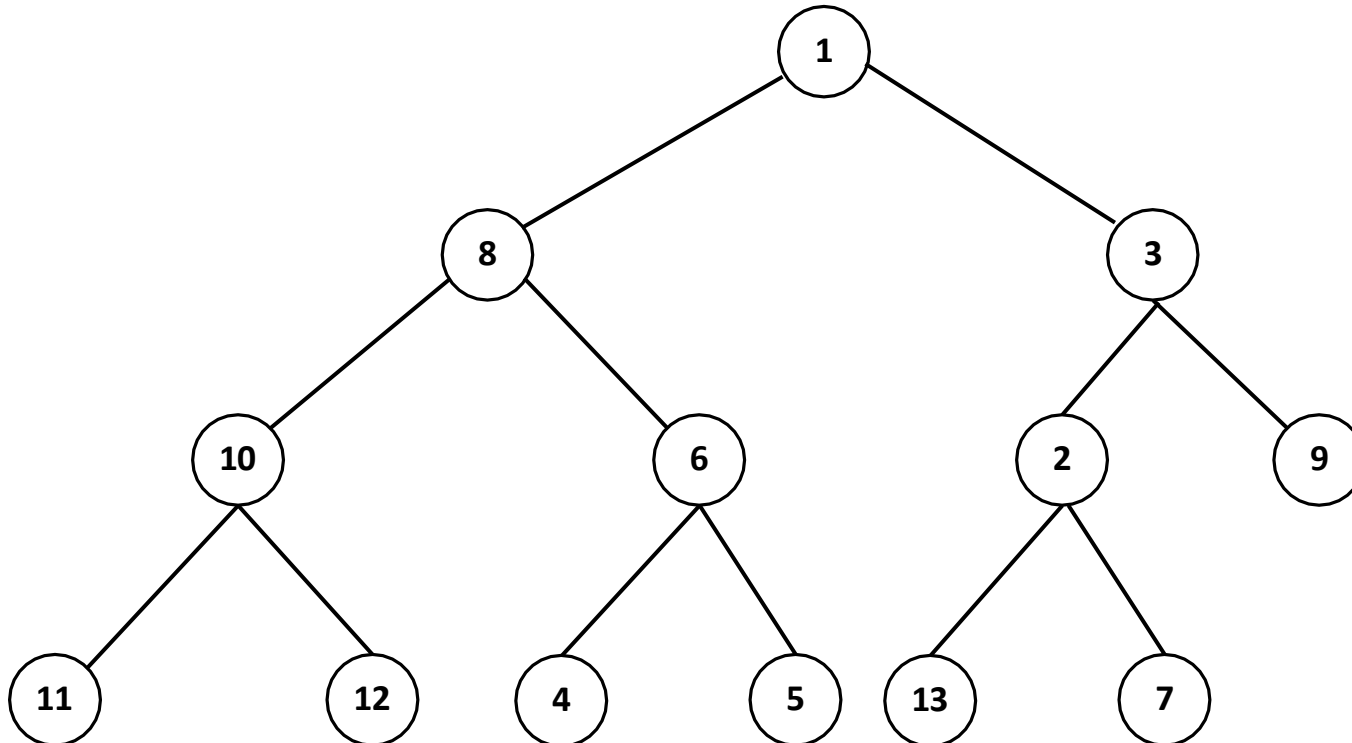
- 배열을 입력으로 받아 힙 구조로 만들기

1	8	3	10	6	2	9	11	12	4	5	13	7
---	---	---	----	---	---	---	----	----	---	---	----	---

Build Heap: sift down

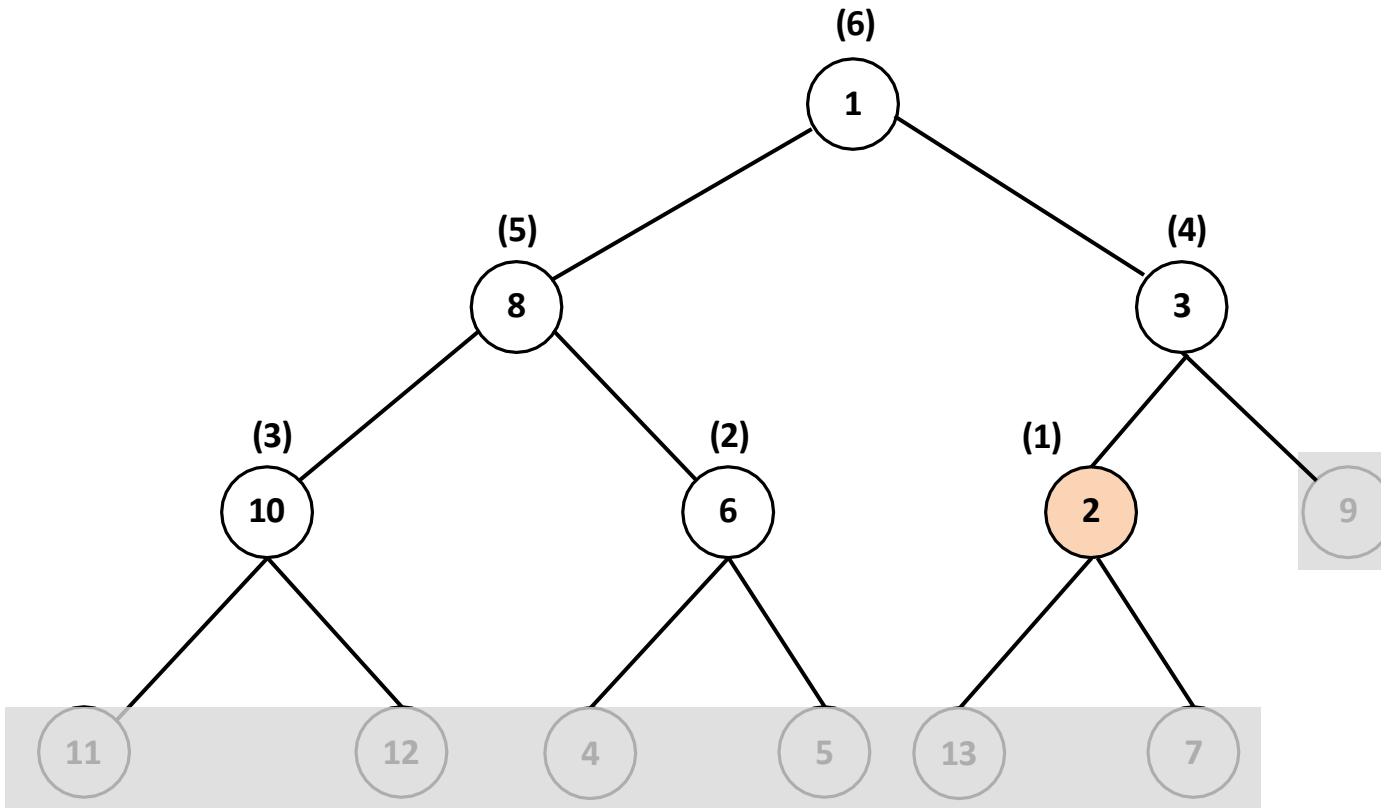
- 배열을 입력으로 받아 힙 구조로 만들기

1	8	3	10	6	2	9	11	12	4	5	13	7
---	---	---	----	---	---	---	----	----	---	---	----	---



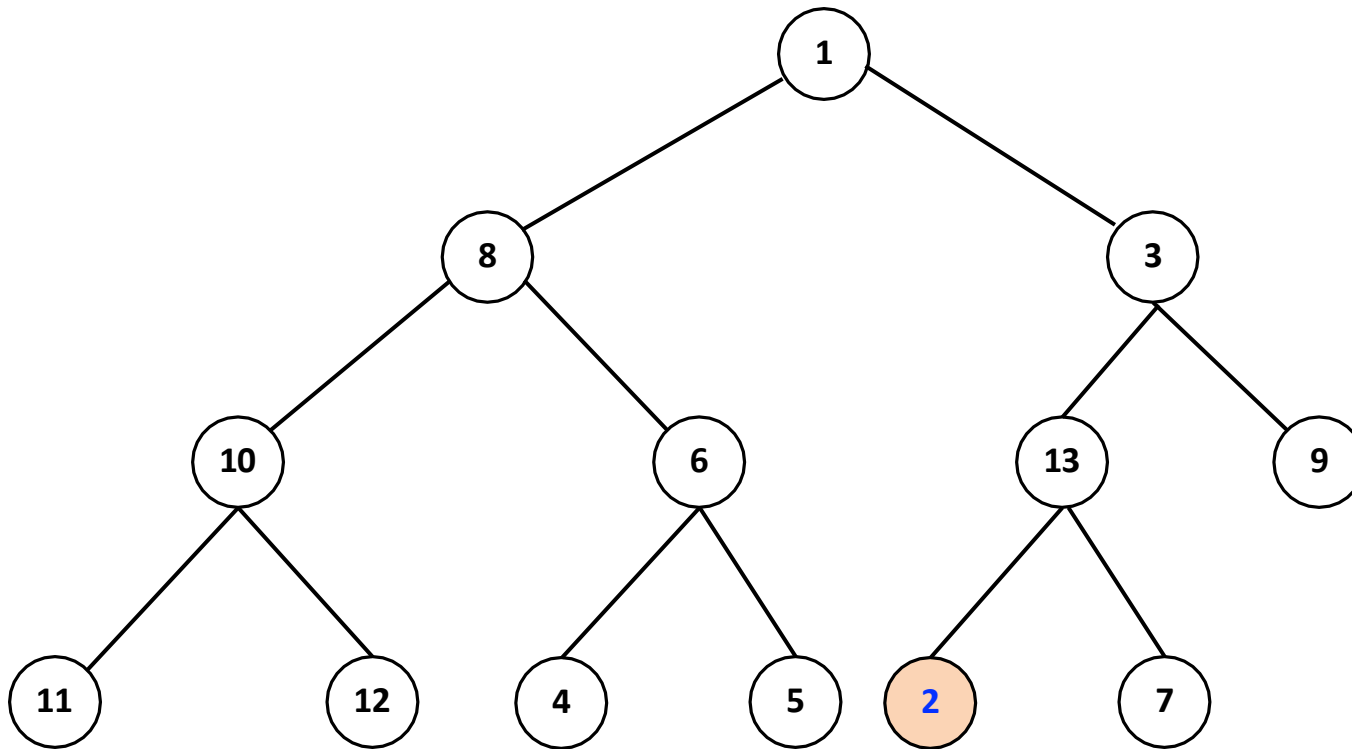
Build Heap: sift down

- 마지막 내부 노드부터 루트까지를 대상으로, 차례대로 sift down 연산 수행
 - Sift down: 자신의 자식보다 우선순위가 낮을 경우, 자식과의 위치 교환



Build Heap: sift down

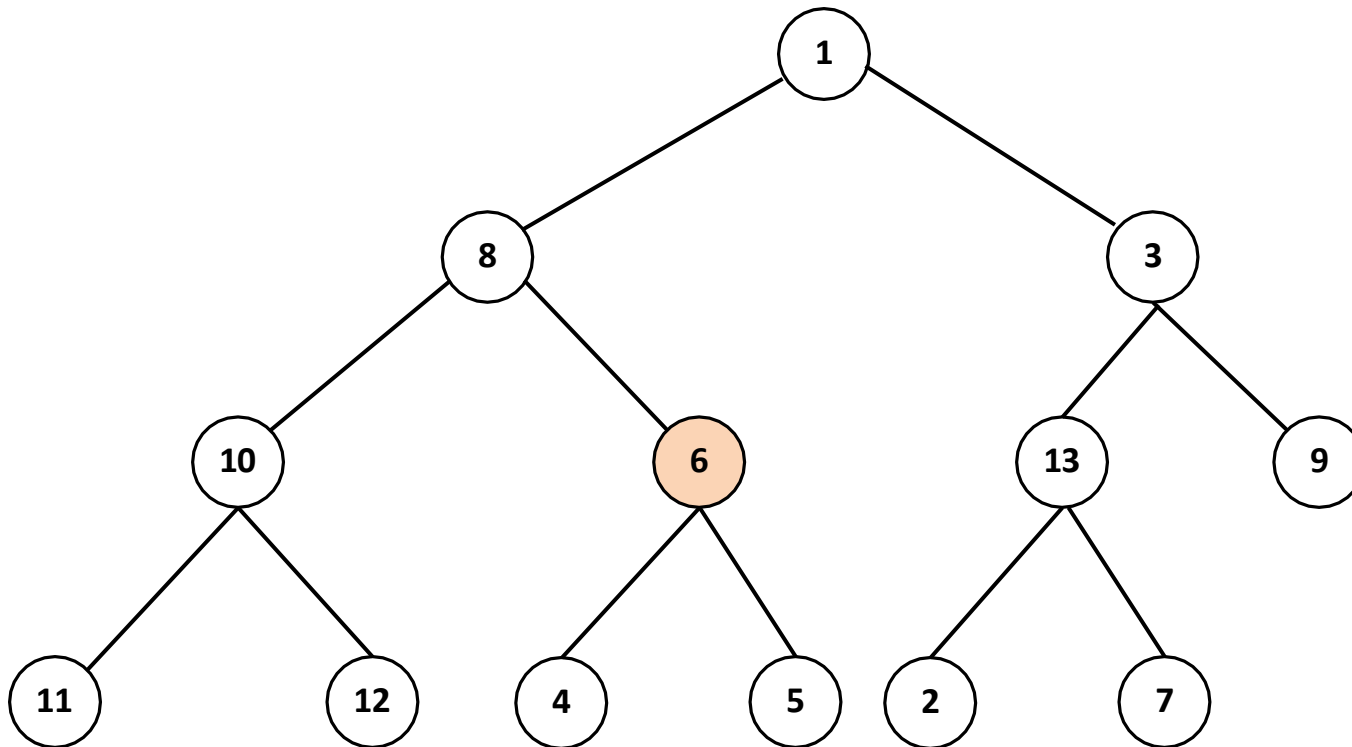
- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



노드 2는 자신의 자식들 중 더 큰 자식인
노드 13과 자리를 교환

Build Heap: sift down

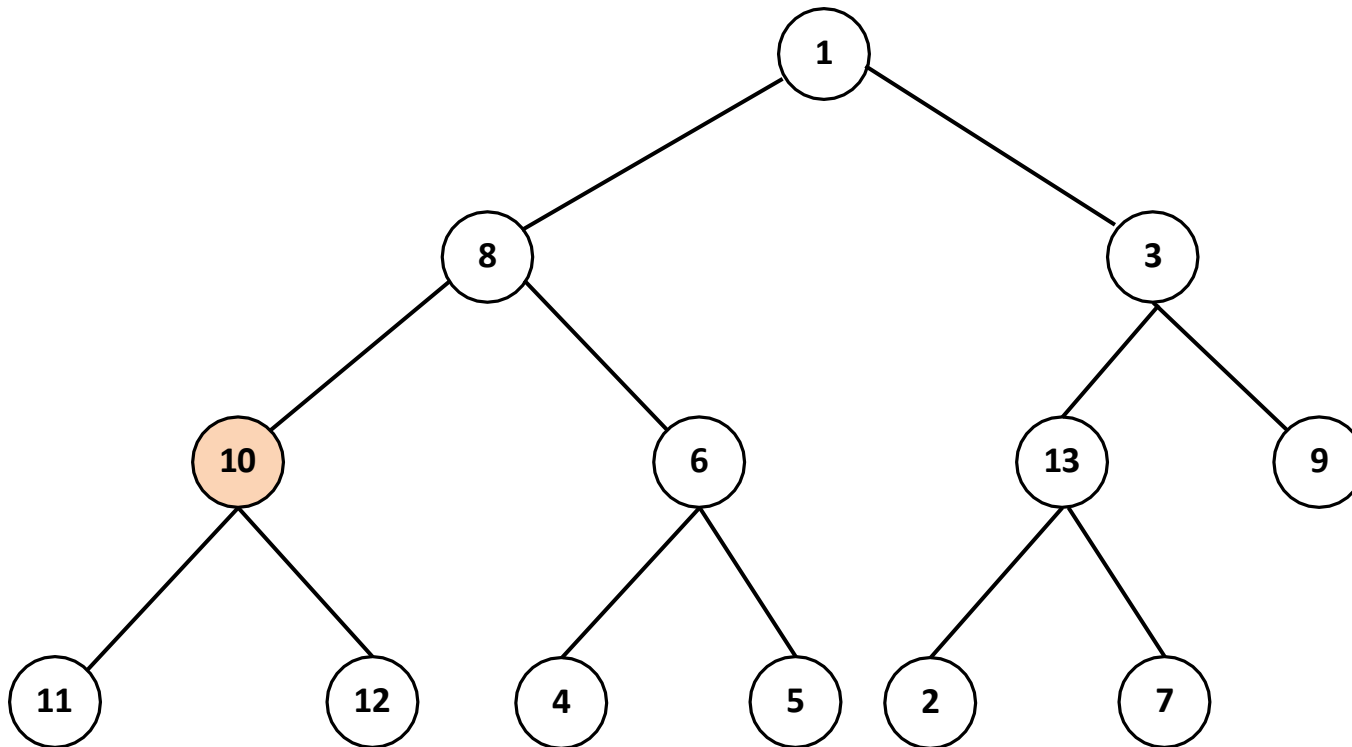
- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



노드 6은 이미 자신의 자식들보다 더 크므로,
교환 일어나지 않음

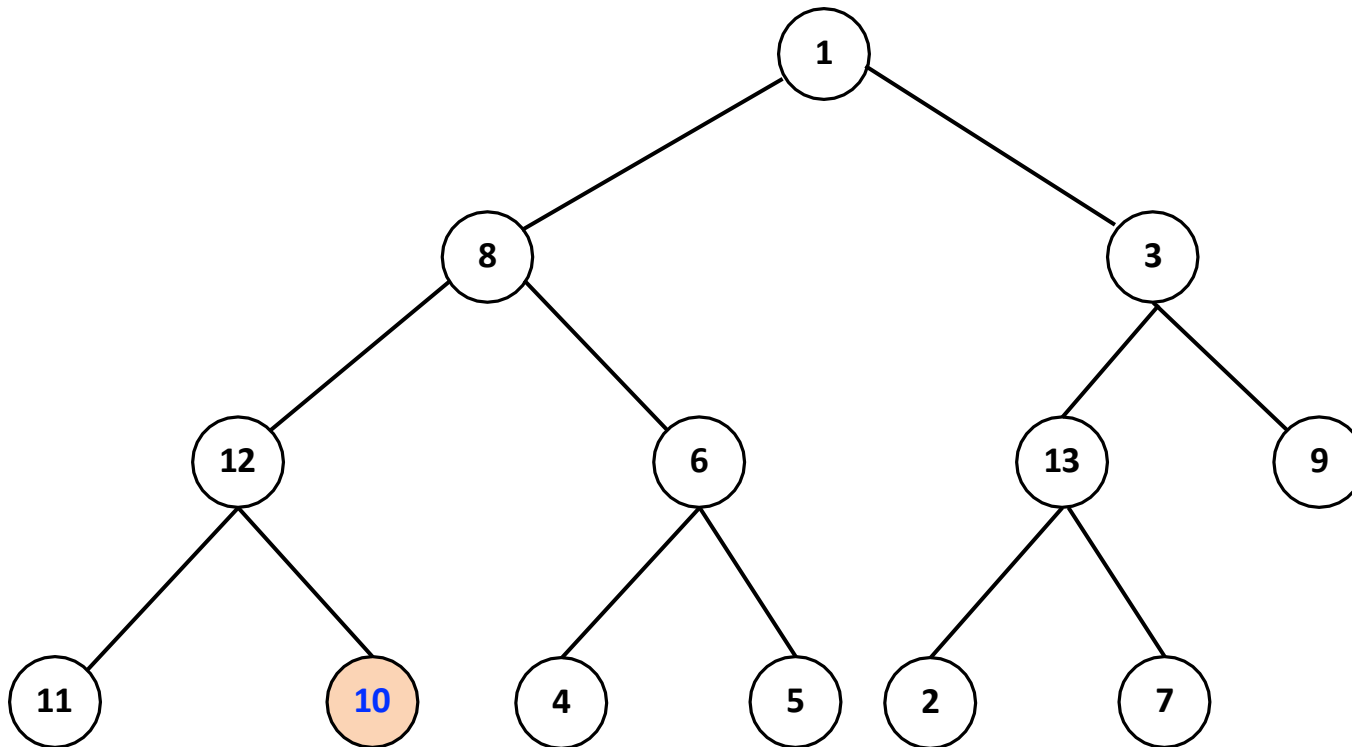
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



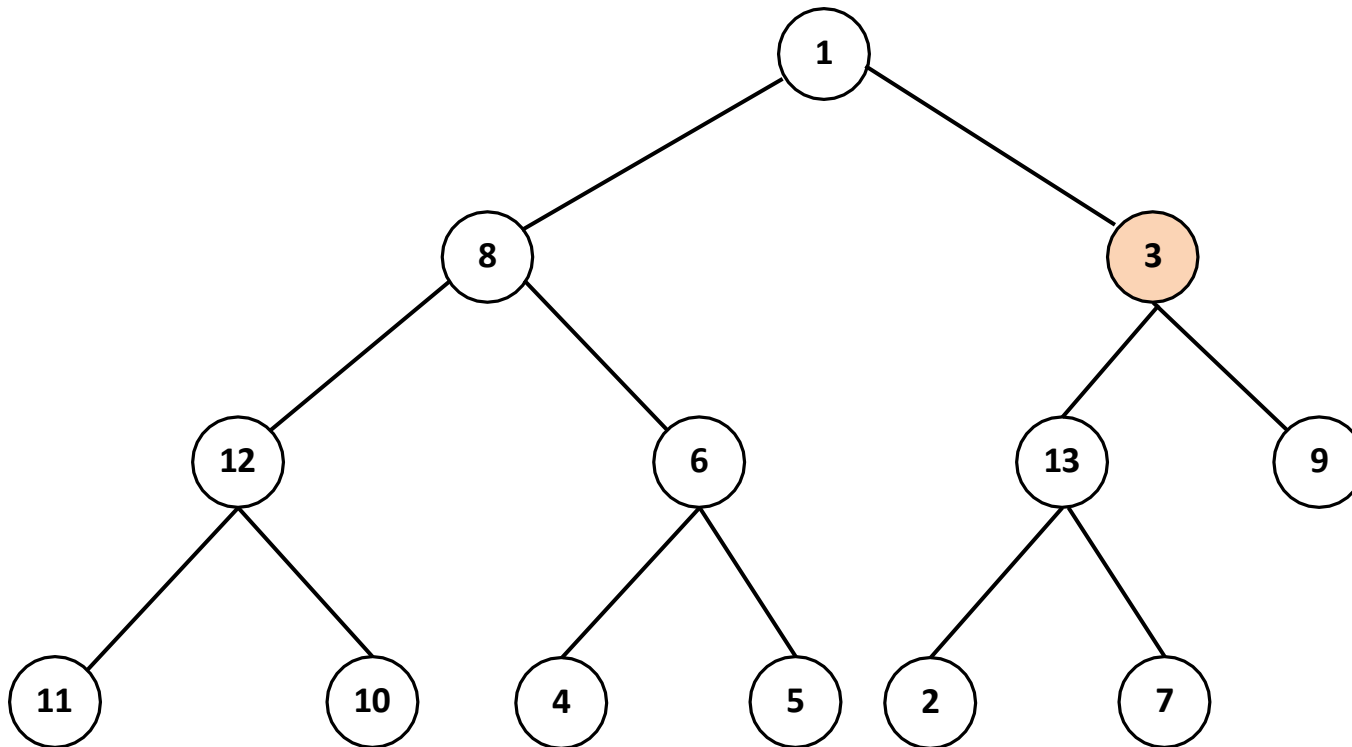
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



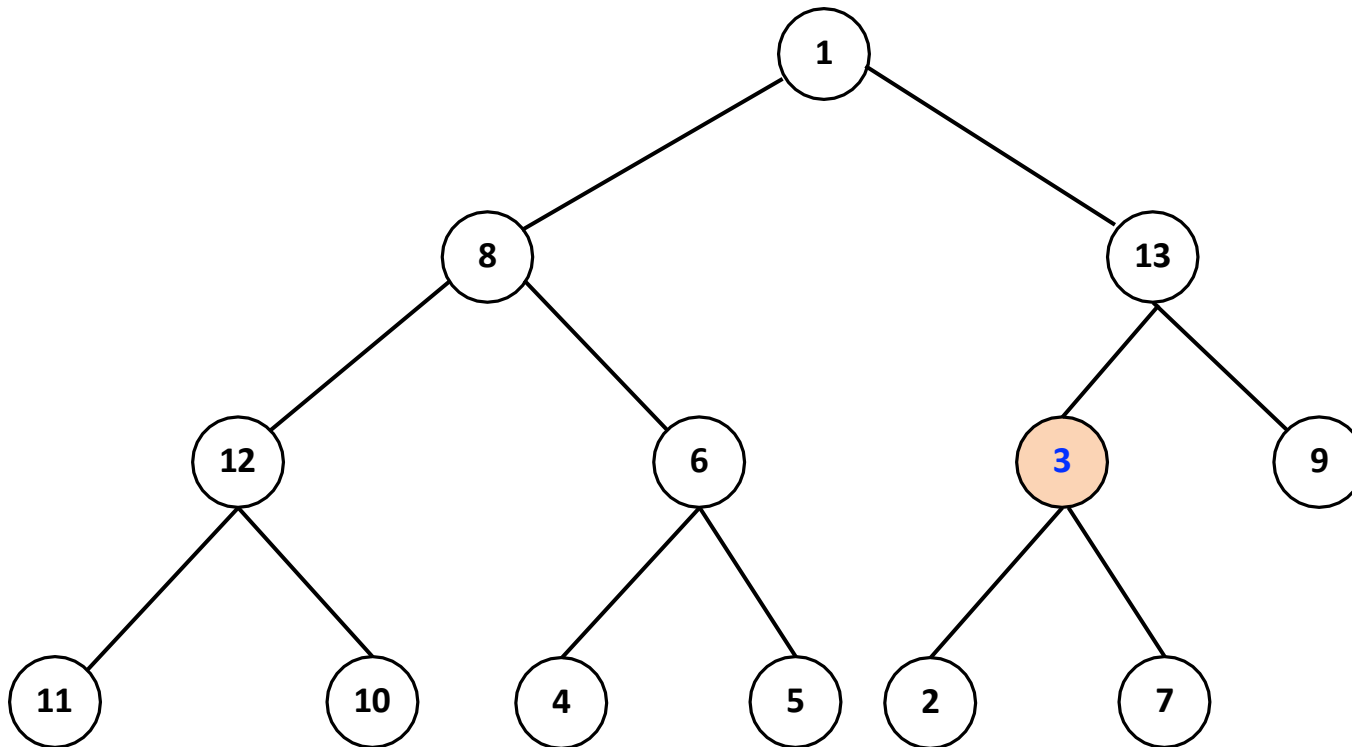
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



Build Heap: sift down

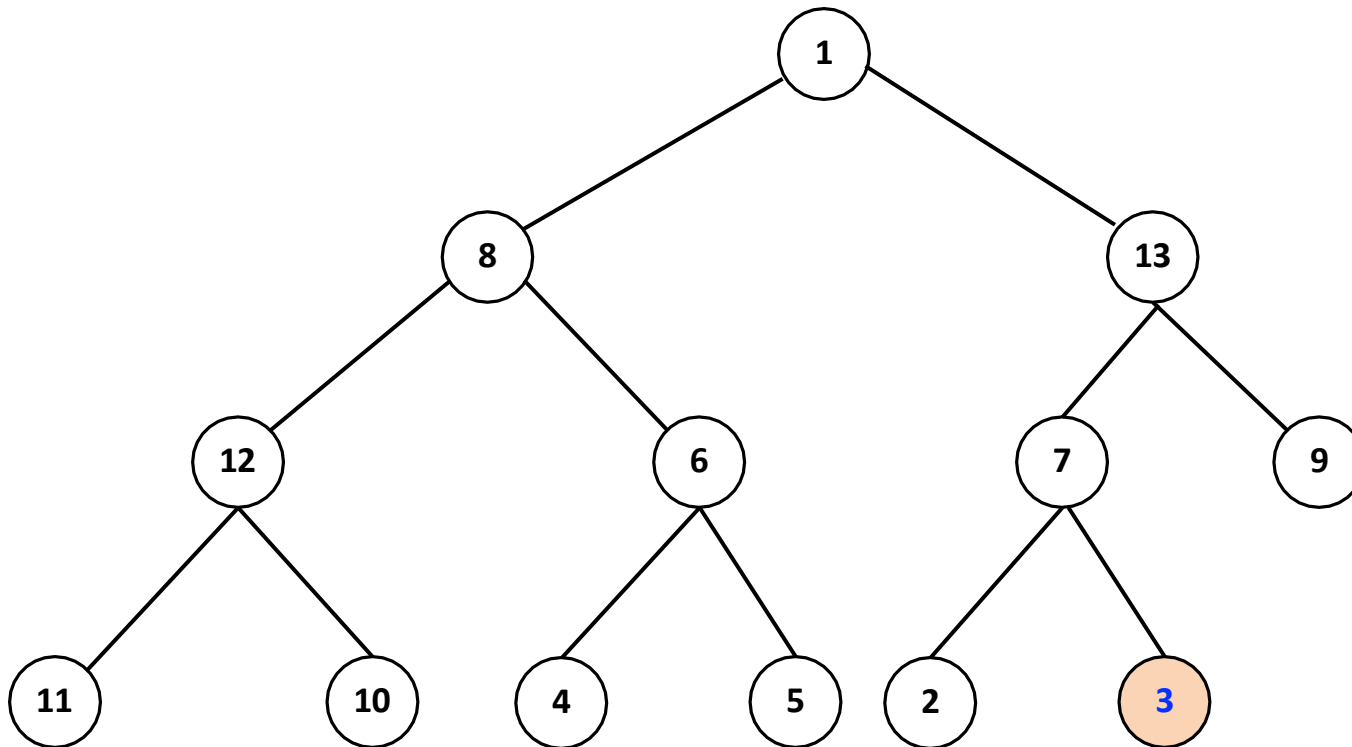
- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



노드 3보다 여전히 더 큰 자식 노드가 존재

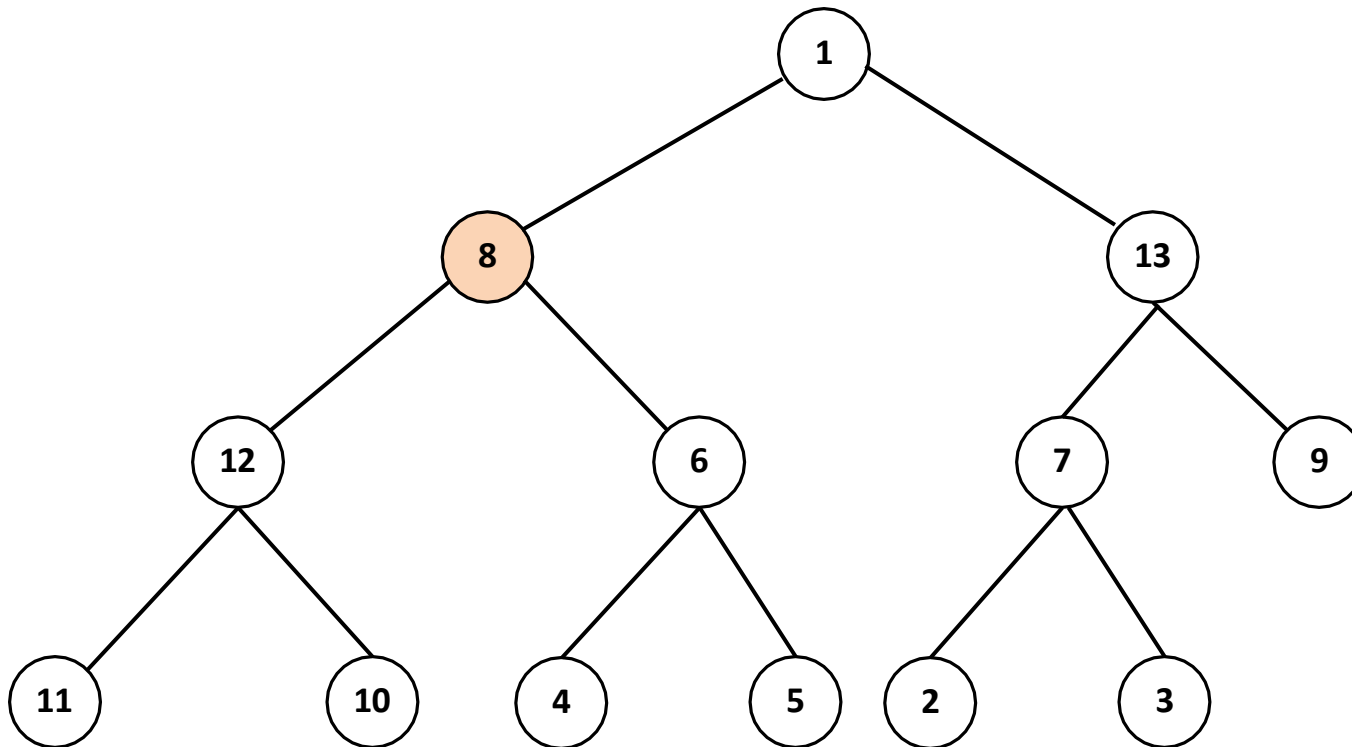
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



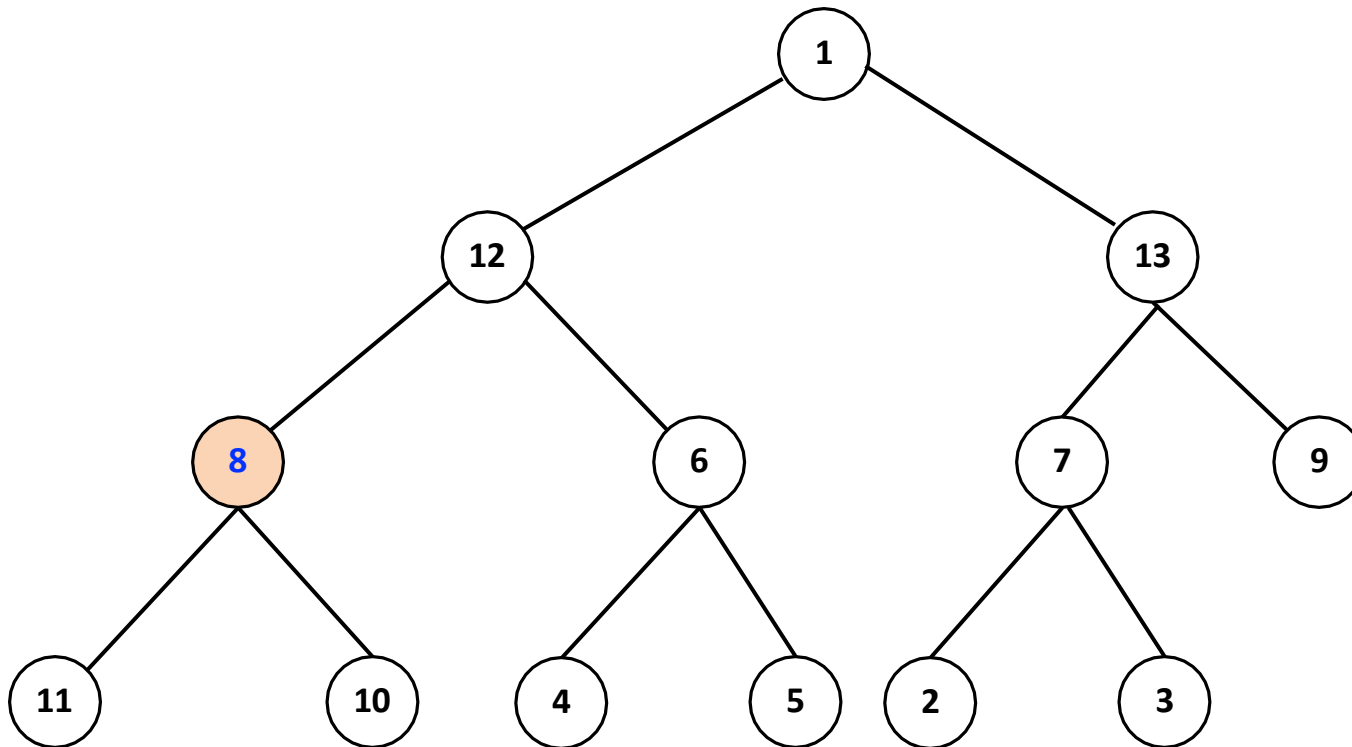
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



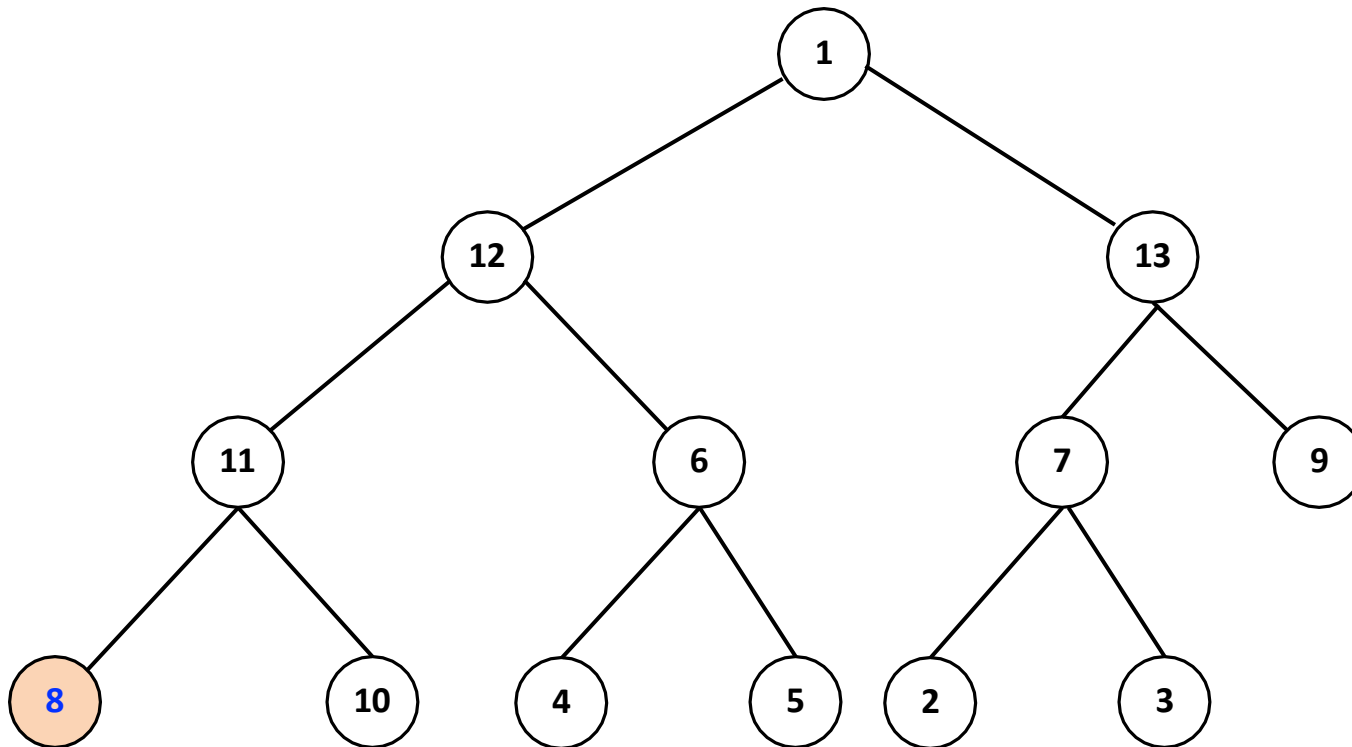
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



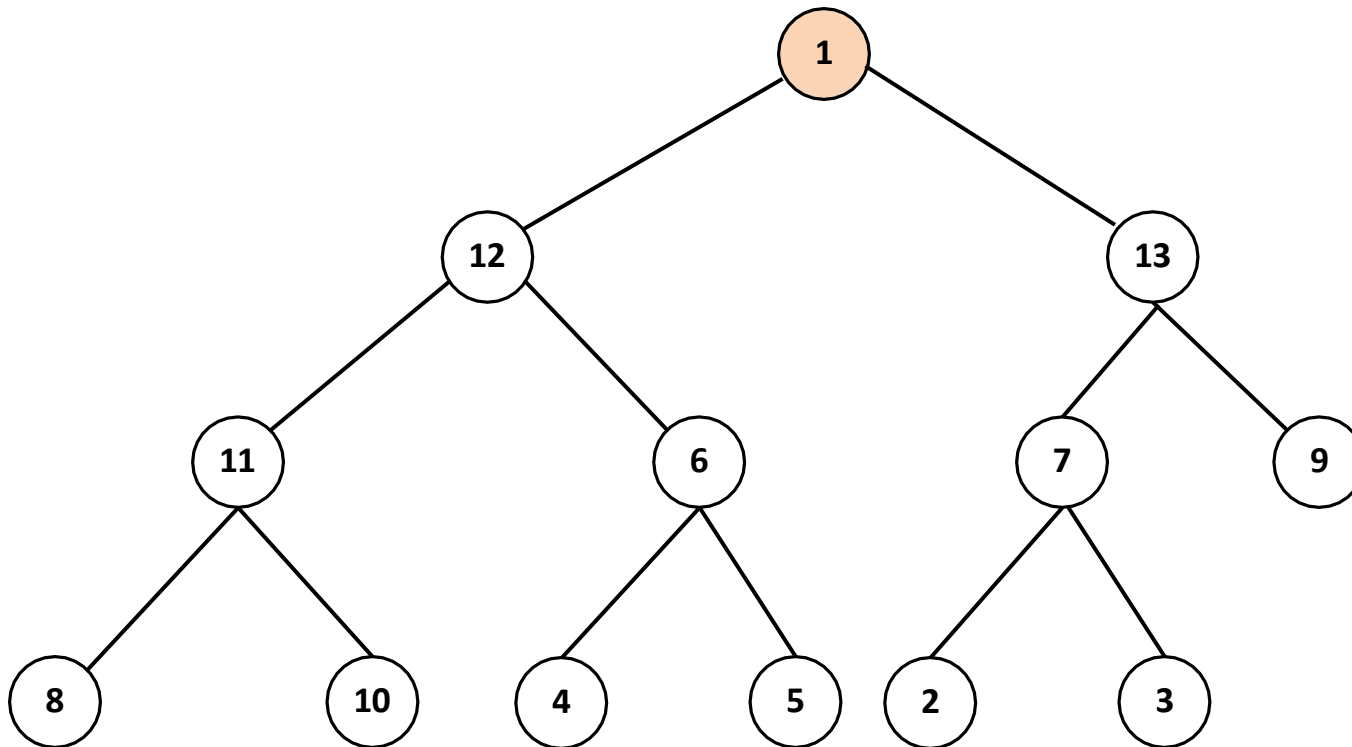
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



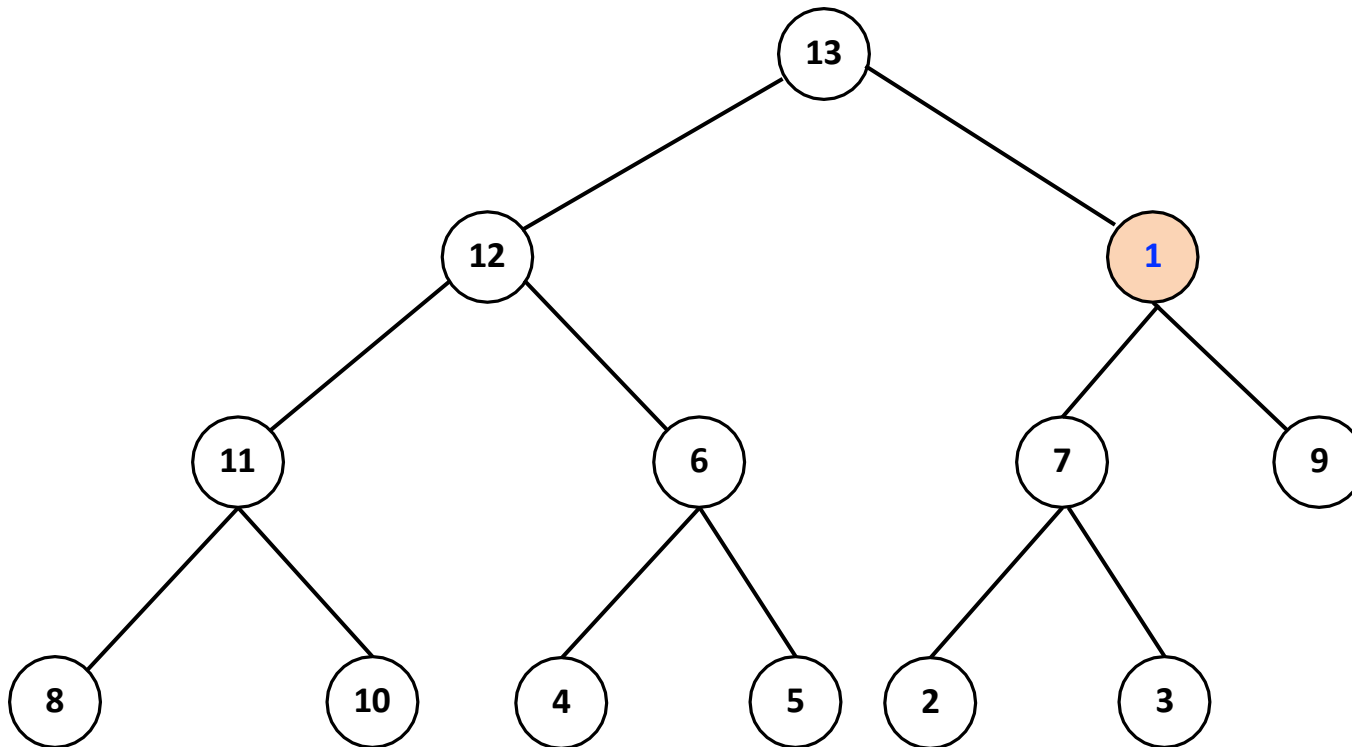
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환



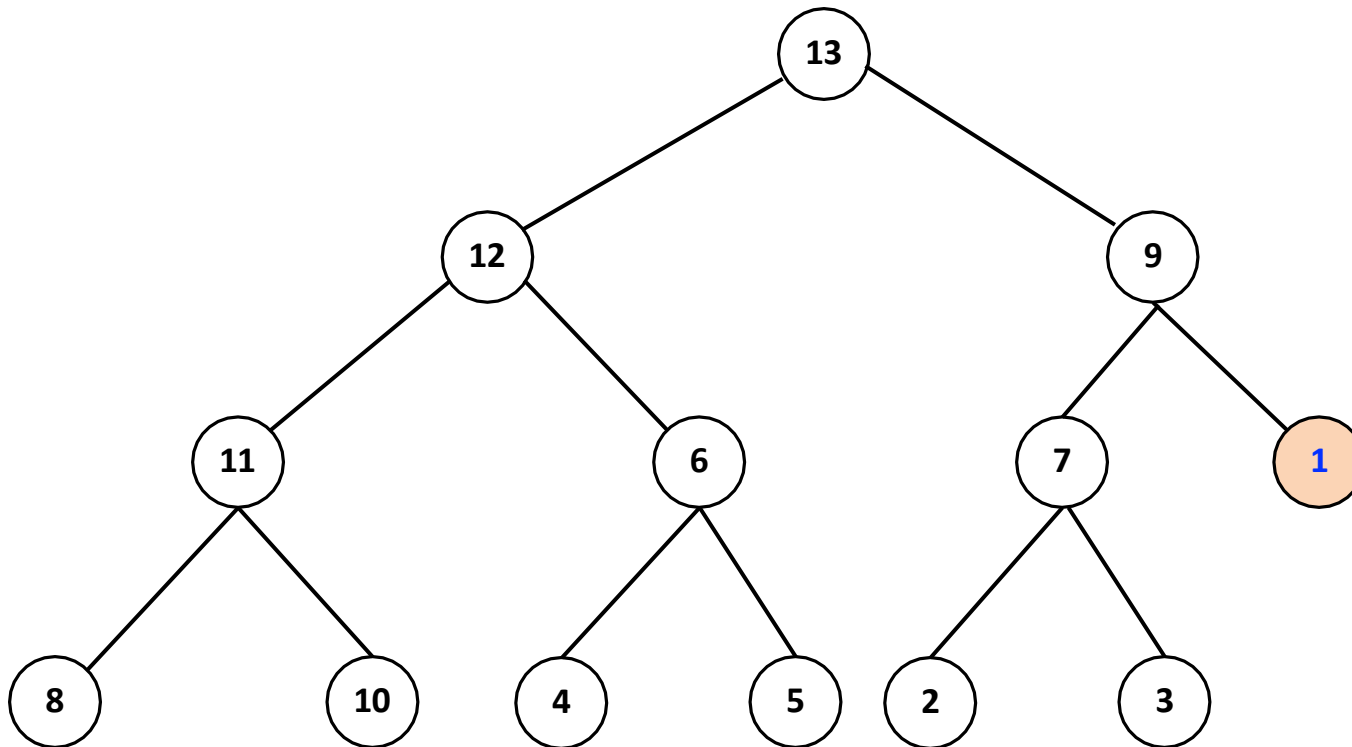
Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환

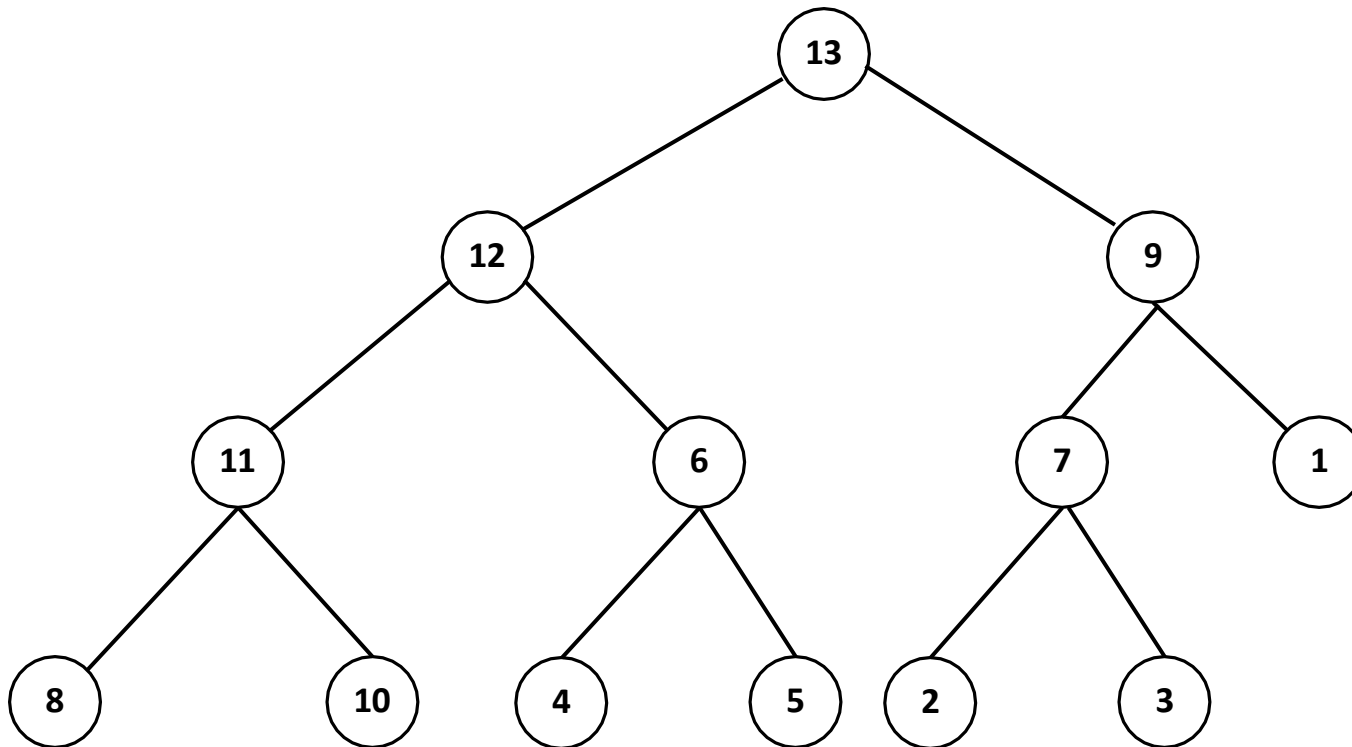


Build Heap: sift down

- Sift down: 자식보다 우선 순위가 낮을 경우, 더 큰 자식과 교환

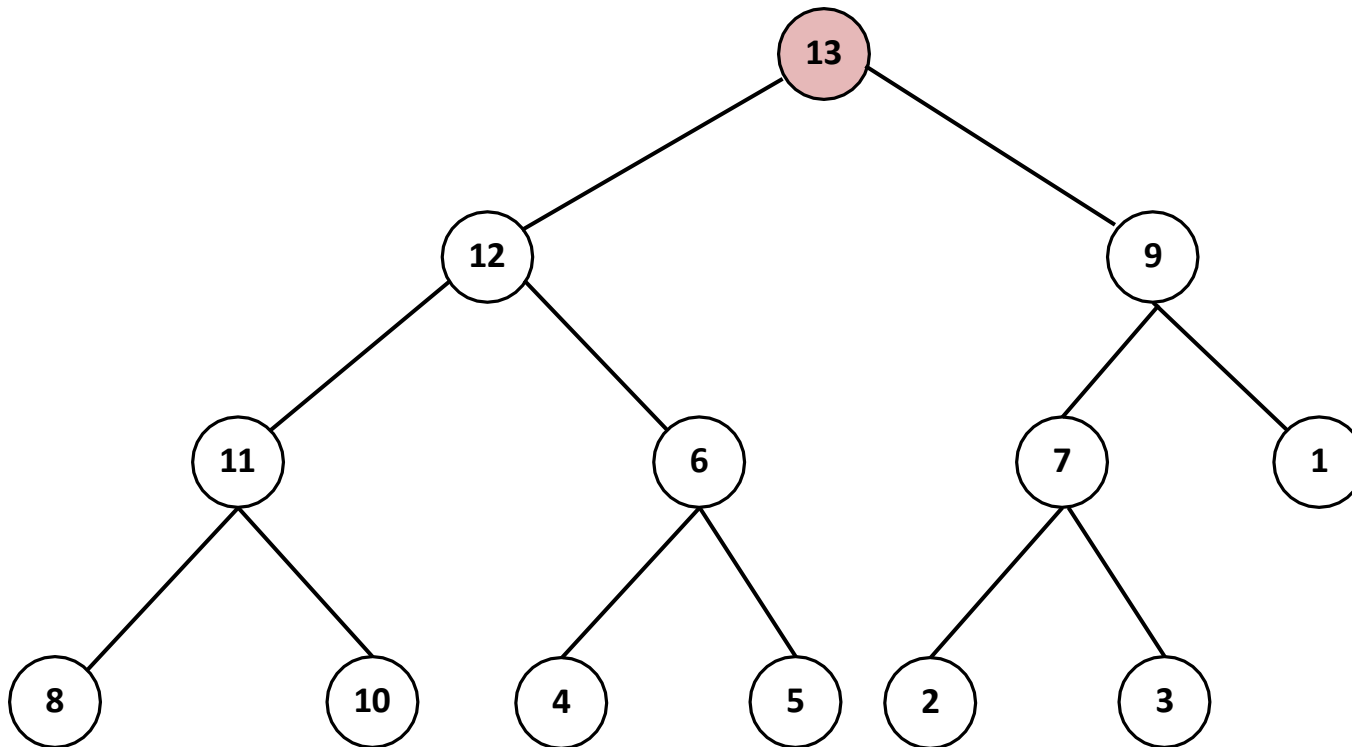


Build Heap: 완성



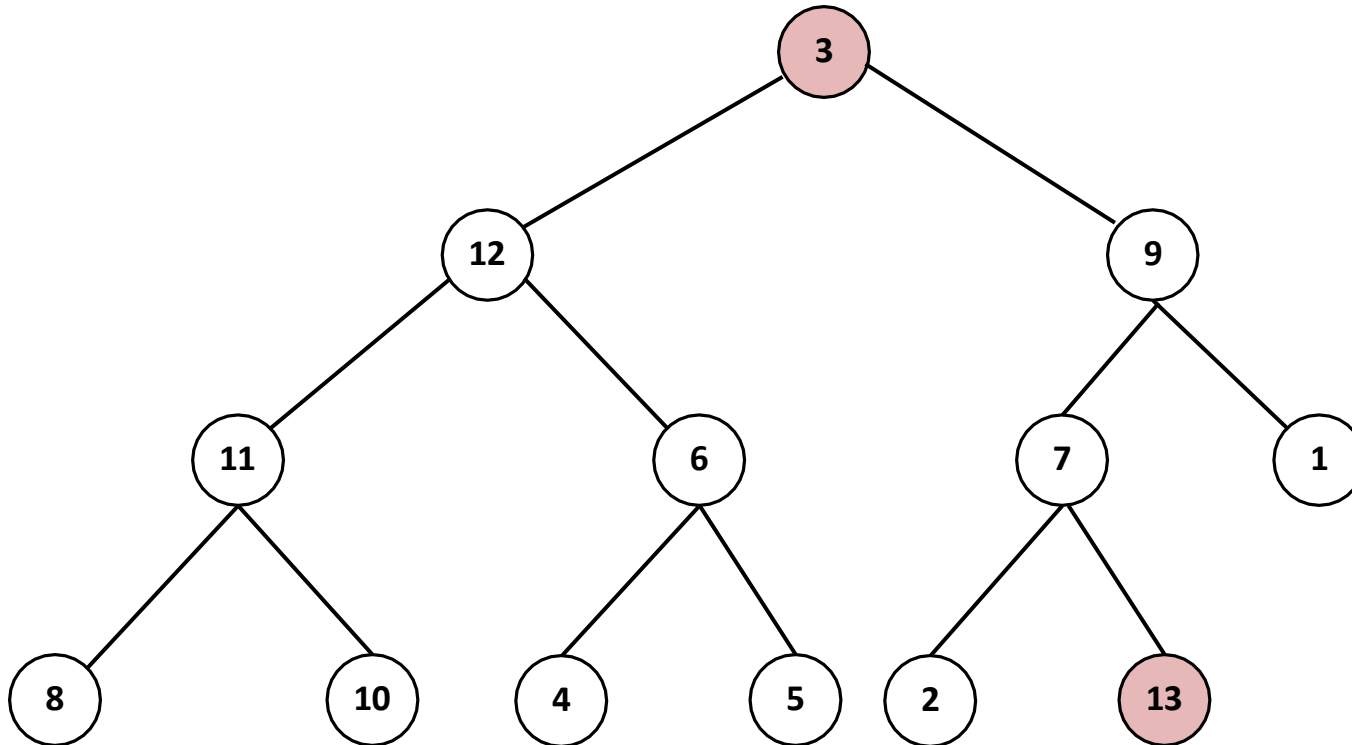
Remove Max

- 힉의 최대값(우선순위가 가장 높은 값)을 반환하는 작업



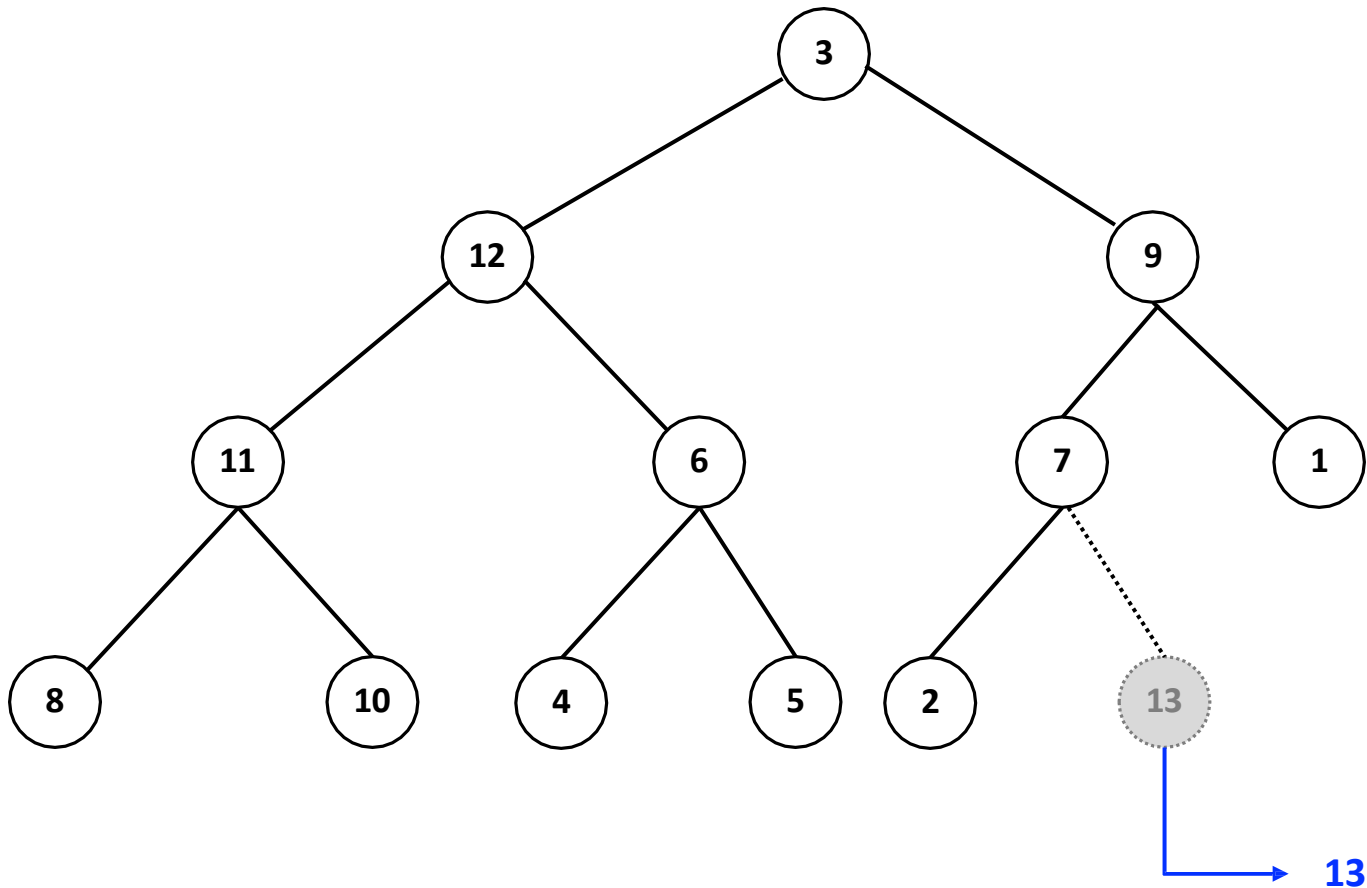
Remove Max

Step 1. 마지막 노드와 루트 노드 간의 위치를 서로 바꿈



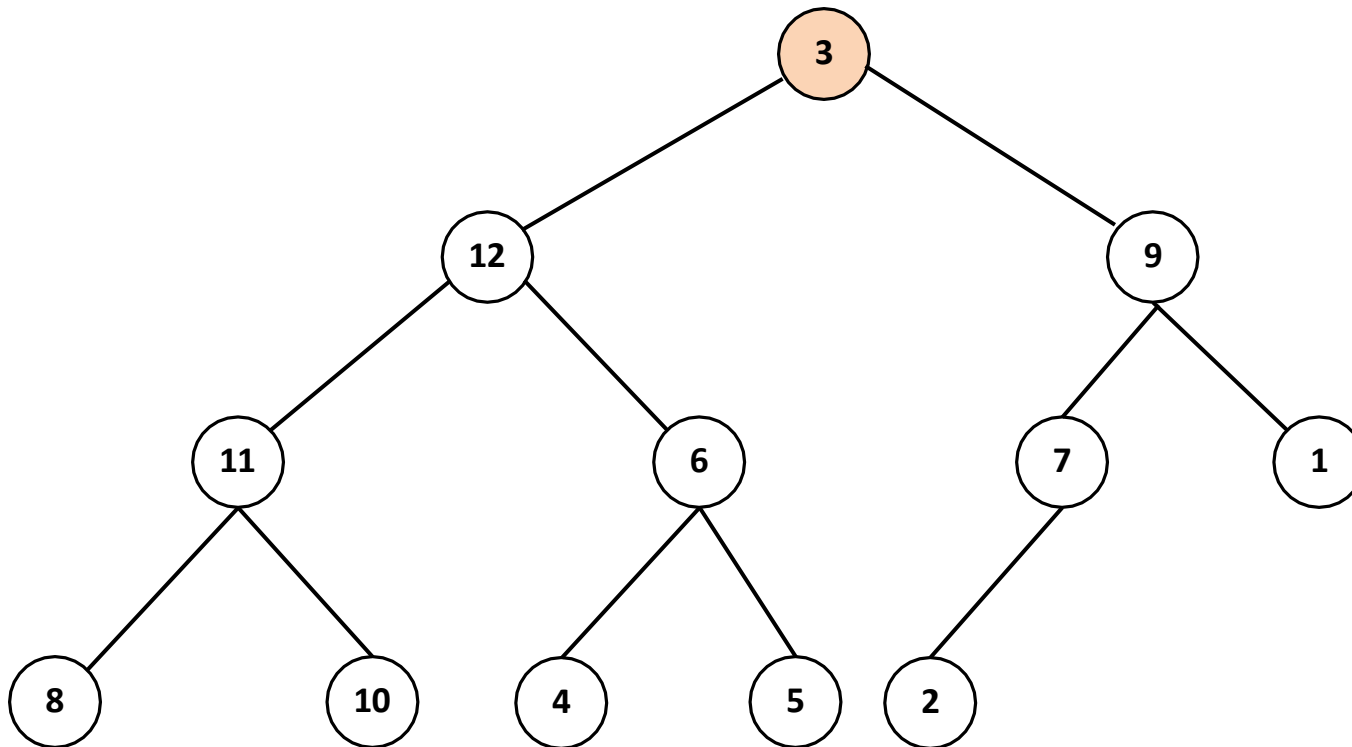
Remove Max

Step 2. 마지막 노드 제거 (가장 우선순위가 높은 값 반환)



Remove Max

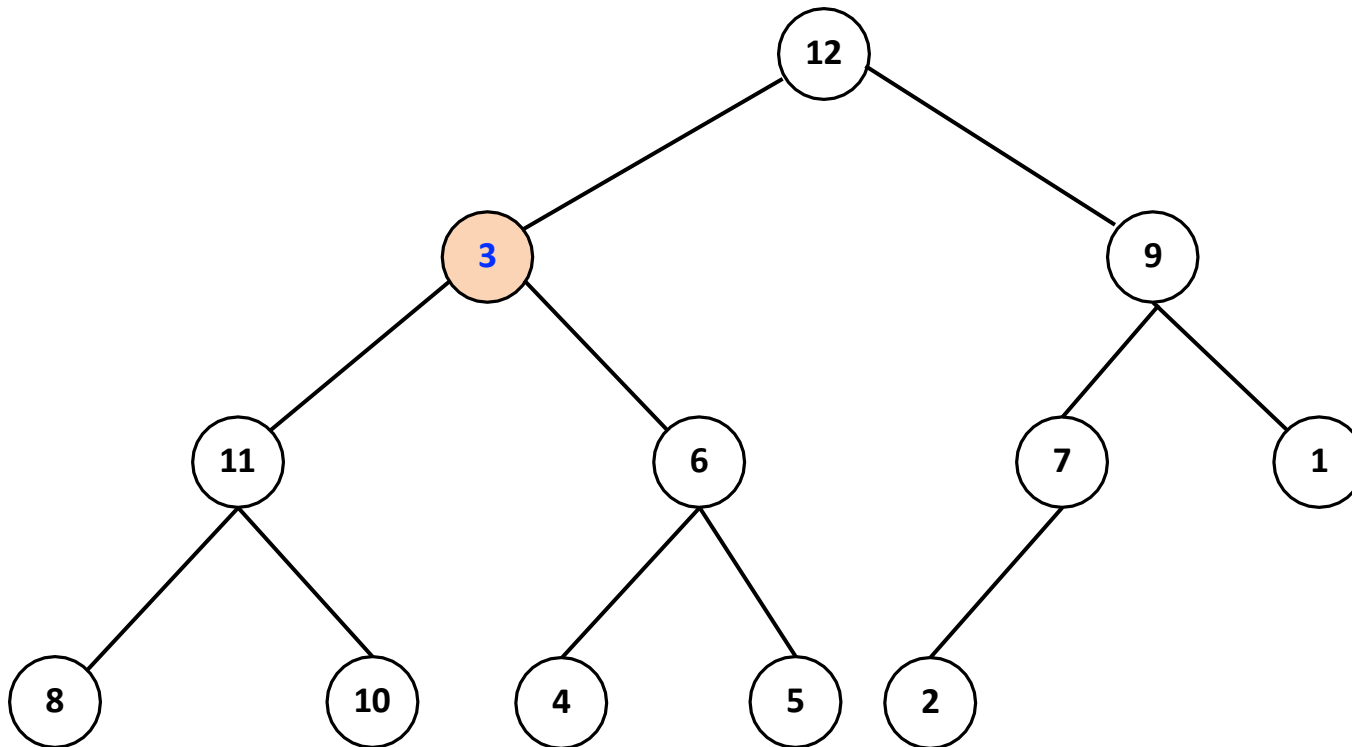
Step 3. 새 루트에서부터 sift down 수행



13

Remove Max

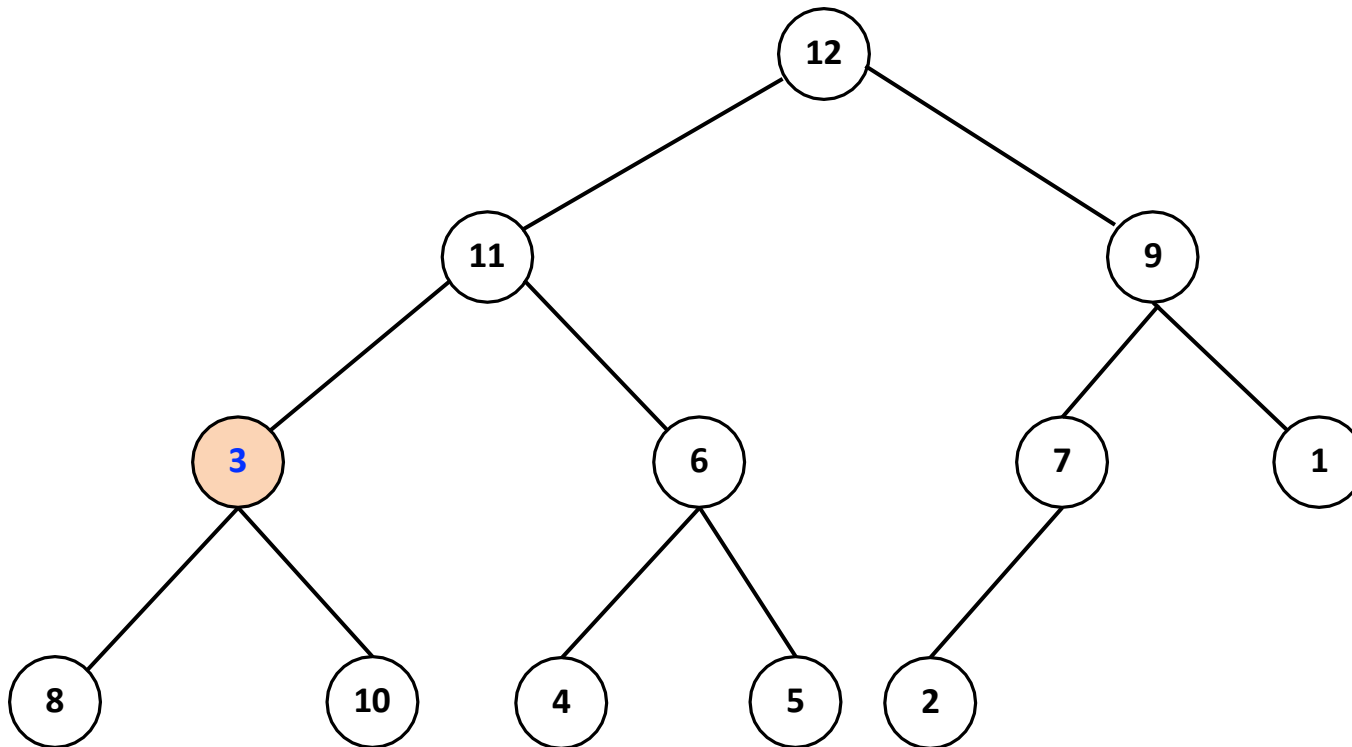
Step 3. 새 루트에서부터 sift down 수행



13

Remove Max

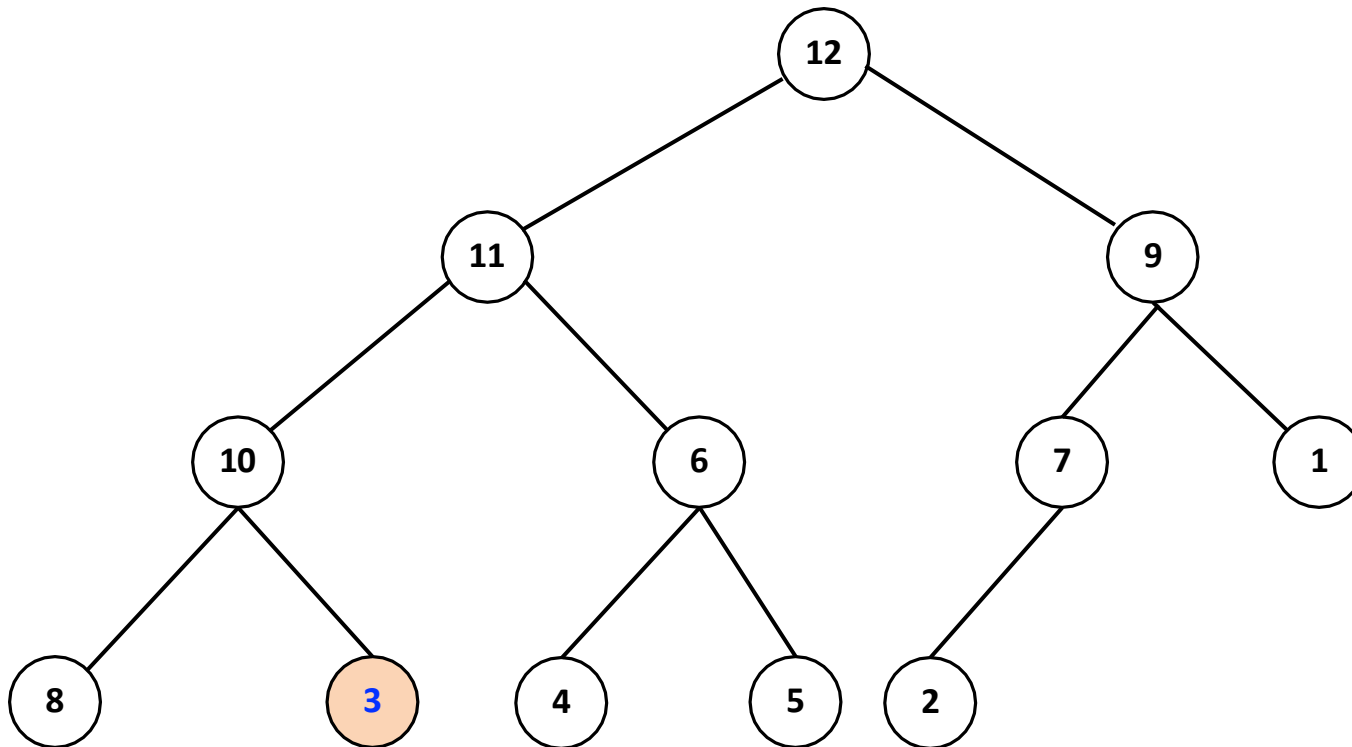
Step 3. 새 루트에서부터 sift down 수행



13

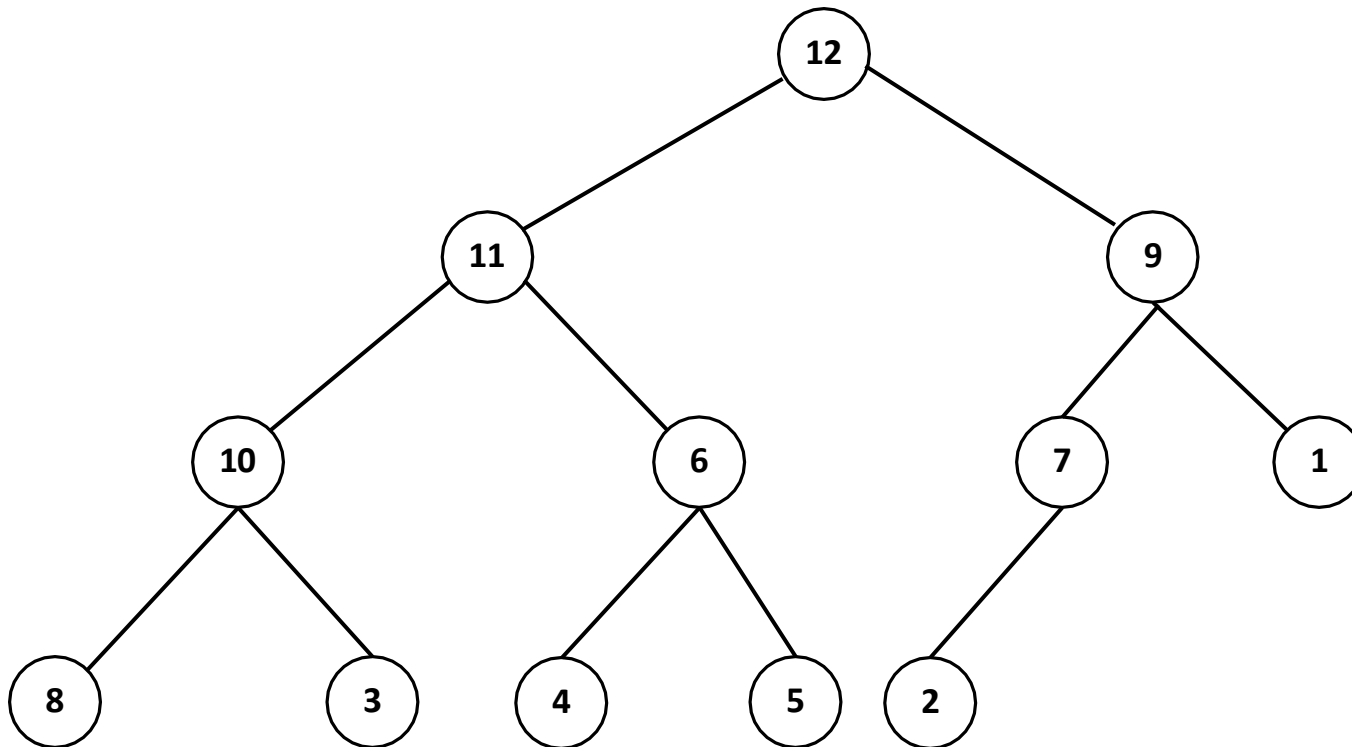
Remove Max

Step 3. 새 루트에서부터 sift down 수행



13

Remove Max: 완성



구현: sift down

```
private void siftDown(int pos) {  
    while (!isLeaf(pos)) {  
        int largest = leftChild(pos); // 포지션 (인덱스) 저장  
        int rc = rightChild(pos);  
  
        // 두 자식 중 더 큰 값의 자식 찾기  
        if (rc < n && Heap[rc] > Heap[largest])  
            largest = rc;  
  
        // 현재 노드가 자식보다 이미 크다면 추가적인 처리 없이 함수 종료  
        if (Heap[pos] >= Heap[largest])  
            return;  
  
        swap(pos, largest); // 현재 노드와 자식 노드 간의 위치 교환  
        pos = largest;      // 자식 노드의 인덱스를 현재 노드의 인덱스(pos)에 저장  
    }  
}
```

Look ahead

Heap Sort